

Entity:
Product

Design for class “Product”

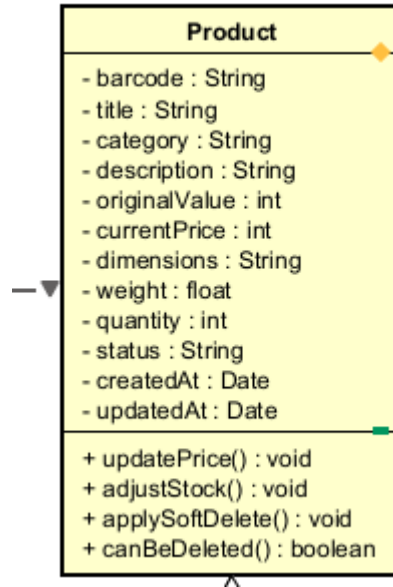


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	barcode	String		Unique identifier for the product (numeric only).
2	title	String		The name of the media product; cannot be blank
3	category	String		Classification: Books, Newspapers, CDs, or DVDs
4	description	String		General description (condition, color, or return terms).
5	originalValue	Integer		Base price excluding 10% VAT; must be a positive integer
6	currentPrice	Integer		Selling price (excl. VAT); must be between 30% and 150% of original value

7	dimensions	String		Physical size (Height x Width x Length); positive values only.
8	weight	Float		Physical weight (kg) used for shipping fee calculation.
9	quantity	Integer		Current inventory count; automatically updated upon sale.
10	status	String	"available"	Commercial availability: "available" or "deactivated".
11	createdAt	DateTime	Current Date	Timestamp when the product record was first created.
12	updatedAt	DateTime	Current Date	Timestamp of the most recent modification to the product data.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	updatePrice (newPrice: Integer)	void	Updates price if it meets the 30-150% rule.
2	adjustStock (quantity: Integer)	void	Modifies quantity (e.g., +50 for new stock).
3	applySoftDelete()	void	Changes status to "deactivated" if stock > 0.
4	canBeDeleted()	Boolean	Returns true only if stock is exactly zero.

1. Operation: updatePrice

This method enforces the pricing policy where the selling price must stay within a specific range relative to the cost.

- Parameter:
 - newPrice: N/A, the proposed new selling price excluding 10% VAT.
- Exception:
 - InvalidPriceRuleException if newPrice is < 30% or > 150% of the originalValue.
- Method:
 - The method calculates the allowed bounds: Lower = originalValue * 0.3 and Upper = originalValue * 1.5.
 - It compares the newPrice parameter against these bounds.
 - If valid, it updates the currentPrice attribute and sets the updatedAt timestamp.
 - If invalid, it throws the exception to prevent the ProductService from proceeding with the update.

2. Operation: adjustStock

This method handles manual inventory changes, such as adding new items or recording damages.

- Parameter:
 - quantity: N/A, the integer amount to change (positive for restock, negative for damage/loss).
- Exception:
 - NegativeStockException if the resulting quantity would be less than zero.
- Method:
 - The method calculates the new quantity by adding delta to the current quantity attribute.
 - It validates that the result is non-negative, as AIMS deals with physical media.
 - If valid, it updates the quantity attribute and triggers an update to the updatedAt field.

3. Operation: applySoftDelete

This method implements the business rule regarding product removal based on current stock levels.

- Parameter:
 - None.
- Exception:
 - IllegalStateException if the product is already in the "deactivated" state.
- Method:
 - The method checks the quantity attribute.
 - If quantity > 0, it changes the status attribute to "deactivated" so the product is no longer available for sale but remains in the database for history.
 - If quantity == 0, it confirms the product is eligible for full deletion.

4. Operation: canBeDeleted

A helper method used by the service to verify if a physical delete is permitted.

- Parameter:
 - None.

- Exception:
 - None.
- Method:
 - It accesses the quantity attribute.
 - Returns true if quantity == 0, otherwise returns false.

Design for class “PrintableProduct”

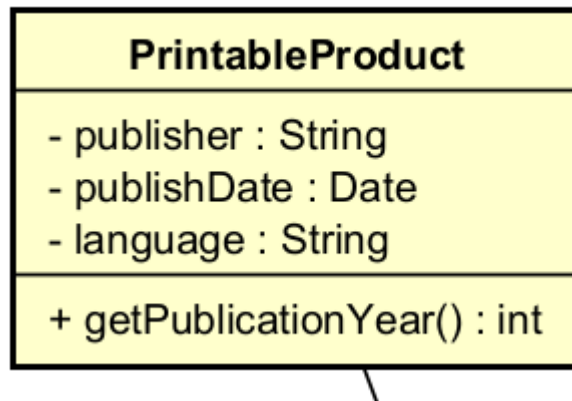


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	publisher	String		The company responsible for publishing.
2	publishDate	Date		The official date of publication.
3	language	String		The language of the content.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	getPublicationYear	int	Extracts the year from <code>publishDate</code> .

Operation: getPublicationYear

- Parameter: None.
- Exception: None.
- Method: * Accesses the publishDate attribute.
 - Returns the year component of the date.

Design for class “DiscProduct”

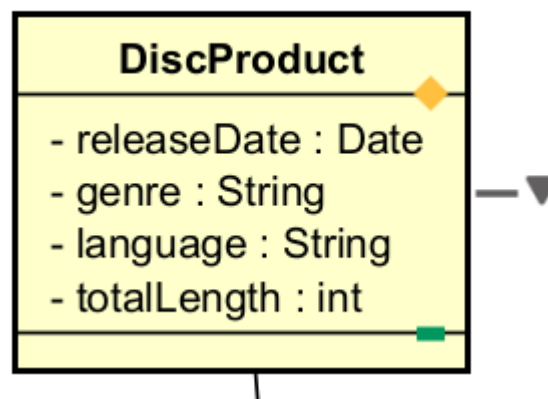


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	releaseDate	Date		The official release date.
2	genre	String		The genre of the content.
3	language	String		The primary language of the disc.
4	totalLength	int		Total duration calculated from components.

Design for class “Book”

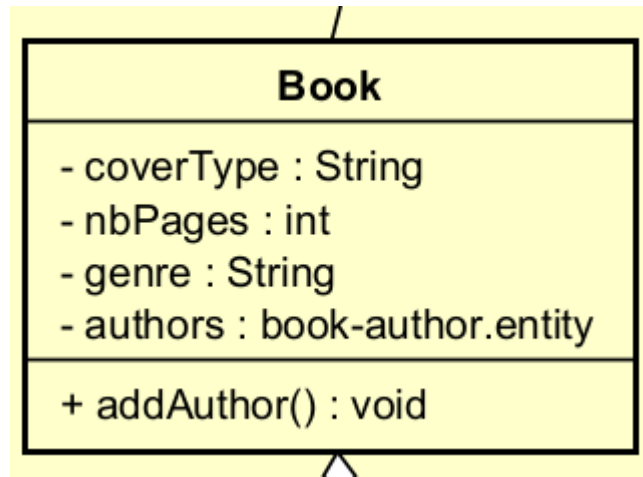


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	coverType	String		"Paperback" or "Hardcover"
2	nbPages	int		Total number of pages in the book.
3	genre	String		The genre of the content.
4	authors	BookAuthor		List of authors

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	addAuthor(author: BookAuthor)	void	Add the author to the authors list.

Design for class “Newspaper”

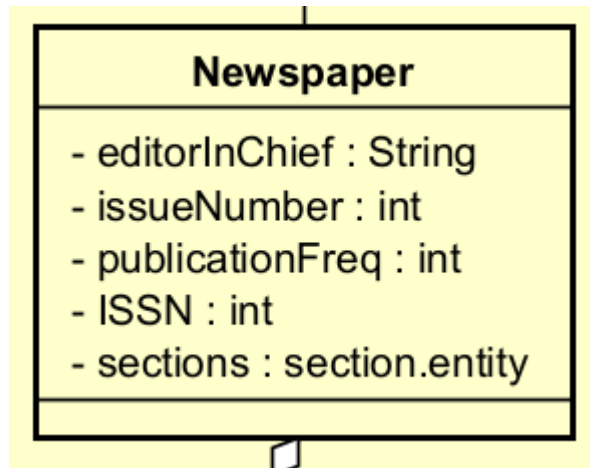


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	editorInChief	String	""	Name of the editor-in-chief of the newspaper publication.
2	issueNumber	int	0	Issue number.
3	publicationFrequency	int	0	Publication frequency (e.g., number of days between issues).
4	ISSN	String	""	International Standard Serial Number.
5	sections	String	""	Main sections of the newspaper (e.g., politics, business, sports, culture).

Design for class “CD”

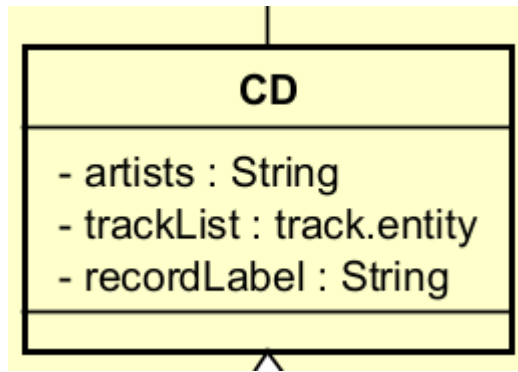


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	artists	String	""	Performers or singers; mandatory when category is CDs.
2	trackList	track	""	Track list with titles and durations.
3	recordLabel	String	""	recordLabel

Design for class “DVD”

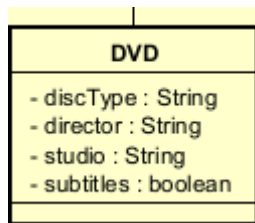


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	discType	String	""	Physical disc type; expected values are Blu-ray or HD-DVD.
2	director	String	""	Director; mandatory when category is DVDs.
3	studio	String	""	Studio name.

4	subtitles	boolean	""	Studio name.
---	-----------	---------	----	--------------

Design for class “ProductHistory”

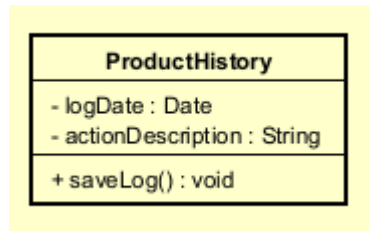


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	logDate	Date	current date/time	Timestamp of the product operation recorded for auditing.
2	actionDescription	String	""	Human-readable description of add, update, delete, deactivate, or quantity adjustment actions.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	saveLog()	void	Persist one history entry after a successful product operation.

Parameter:

- actionDescription: no default value, text created by ProductController to explain what happened.

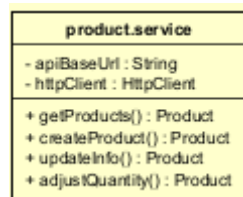
Exception:

- PersistenceException if the history entry cannot be stored.

Method

- saveLog() is invoked after successful Add Product, Update Product, Adjust Quantity, and Delete Product flows.
- The saved description should be specific enough for audit trail requirements in the problem statement.

Design for class “product.service”



Package: client.product-manage.modules.products.services

Role in diagram: Client API service

Design note: Handles HTTP communication between the client product package and the server product package.

Table 1. Attribute design for product.service

#	Name	Data type	Default value	Description
1	apiBaseUrl	string	"/api/products"	Base endpoint used for all product requests.
2	httpClient	HttpClient	injected	Angular HTTP client used to call the server.

Table 2. Operation design for product.service

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	getProducts()	Observable<ProductModel[]>	Retrieves the product list for ProductList component.
2	createProduct(payload)	Observable<ProductModel>	Sends product creation data to the server.
3	updateInfo(id, payload)	Observable<ProductModel>	Updates general product information.
4	adjustQuantity(id, payload)	Observable<ProductModel>	Changes stock quantity through the dialog component.

Parameter:

- id: target product id.
- payload: DTO-like object prepared by ProductForm, UpdateInfo or QuantityDialog.

Exception:

- HTTP/network exception if the server is unavailable or returns an error status.

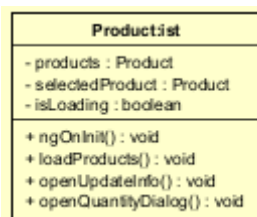
Method

- All client components depend on this service instead of calling server APIs directly.
- Returned observables are subscribed by the UI components.

Other diagrams

- Sequence diagram is useful for request/response interactions between client and server.

Design for class “product-list”



Package: client.product-manage.modules.products.components

Role in diagram: Client list component

Design note: Displays the set of products and delegates item actions to update form or quantity dialog.

Table 1. Attribute design for product-list

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	products	ProductModel[]	[]	Product data rendered in the list screen.
2	selectedProduct	ProductModel null	null	Current product selected by the user.
3	isLoading	boolean	false	Loading flag while retrieving product data.

Table 2. Operation design for product-list

#	Name	Return type	Description (purpose)
1	ngOnInit()	void	Initializes the component and loads product data.
2	loadProducts()	void	Calls ProductService to refresh the list.
3	openUpdateInfo(product)	void	Navigates to update-info component for the chosen product.
4	openQuantityDialog(product)	void	Opens quantity-dialog to edit stock.

Parameter:

- product: selected ProductModel from the rendered list.

Exception:

- View rendering error if product data cannot be mapped to the display model.

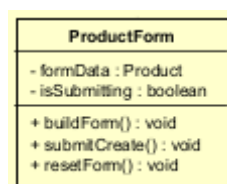
Method

- Acts as the main entry screen of the client products package.
- Delegates all persistence work to client ProductService.

Other diagrams

- A sequence diagram can describe list loading and item-action dispatching.

Design for class “product-form”



Package: client.product-manage.modules.products.components

Role in diagram: Client form component

Design note: Captures data required to create a new product from the client side.

Table 1. Attribute design for product-form

#	Name	Data type	Default value	Description
1	formData	ProductModel	empty model	Editable product data bound to the create form.
2	isSubmitting	boolean	false	Submission state while create request is in progress.

Table 2. Operation design for product-form

#	Name	Return type	Description (purpose)
1	buildForm()	void	Initializes default form values and validators.
2	submitCreate()	void	Collects user input and calls ProductService.createProduct().
3	resetForm()	void	Returns the form to its default state after completion or cancel.

Parameter:

- formData: values entered by the user for a new product.

Exception:

- Validation error if required values are missing before submission.

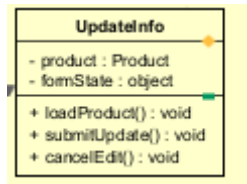
Method

- Used only for creation flow on the client package.
- Server DTO mapping is performed through client ProductService.

Other diagrams

- Activity diagram may be added later for the create-product workflow.

Design for class “update-info”



Package: client.product-manage.modules.products.components

Role in diagram: Client edit component

Design note: Edits general information of an existing product.

Table 1. Attribute design for update-info

#	Name	Data type	Default value	Description
1	product	ProductModel null	null	Target product being edited.
2	formState	object	{}	Temporary editable state for title, type, price and related fields.

Table 2. Operation design for update-info

#	Name	Return type	Description (purpose)
1	loadProduct(id)	void	Loads the current product information into the edit form.
2	submitUpdate()	void	Calls ProductService.updateInfo() with the modified values.
3	cancelEdit()	void	Discards local changes and returns to the previous screen.

Parameter:

- id: product id selected from ProductList.

Exception:

- Validation error if the edit payload is incomplete or inconsistent.

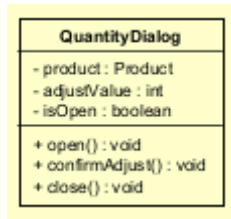
Method

- Reuses product.model as client data carrier.
- Should be reached from ProductList or an equivalent navigation route.

Other diagrams

- Sequence diagram is suitable for edit and save interaction.

Design for class “quantity-dialog”



Package: client.product-manage.modules.products.components

Role in diagram: Client quantity dialog

Design note: Popup/dialog used to increase or decrease product quantity.

Table 1. Attribute design for quantity-dialog

#	Name	Data type	Default value	Description
1	product	ProductModel null	null	Product whose quantity is being changed.
2	adjustValue	number	0	Positive or negative quantity adjustment entered by the user.
3	isOpen	boolean	false	Dialog visibility state.

Table 2. Operation design for quantity-dialog

#	Name	Return type	Description (purpose)
1	open(product)	void	Opens the dialog with the selected product context.
2	confirmAdjust()	void	Sends the adjustment request through ProductService.
3	close()	void	Closes the dialog without changing data.

Parameter:

- product: selected product from ProductList.
- adjustValue: stock change requested by the user.

Exception:

- Validation error if the resulting quantity would become invalid.

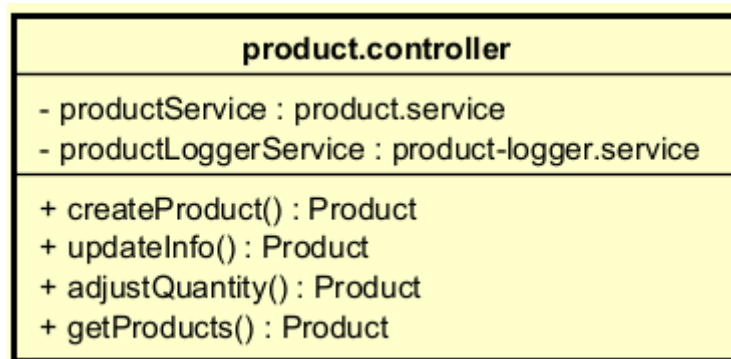
Method

- Should be launched from ProductList and may update the list after a successful response.

Other diagrams

- Sequence diagram can show dialog confirm flow and returned updated quantit

Design for class “product.controller”



Package: server.modules.products.controllers

Role in diagram: Server controller

Design note: Receives HTTP requests and delegates product operations to ProductService.

Table 1. Attribute design for product.controller

#	Name	Data type	Default value	Description
1	productService	ProductService	injected	Business service used to execute product use cases.
2	productLoggerService	ProductLoggerService	injected	Service used to persist history information for product changes.

Table 2. Operation design for product.controller

#	Name	Return type	Description (purpose)
1	createProduct(dto)	Product	Creates a new product using create-product.dto.
2	updateInfo(id, dto)	Product	Updates product information using update-info.dto.

3	adjustQuantity(id, dto)	Product	Adjusts stock quantity using adjust-quantity.dto.
4	getProducts()	Product[]	Returns the current product set to the client.

Parameter:

- id: product id from the HTTP route.
- dto: request body validated by the matching DTO class.

Exception:

- HTTP bad request or not-found exception if the input is invalid or product id does not exist.

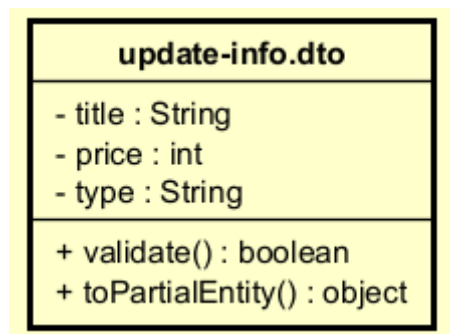
Method

- Thin orchestration layer only; validation and business logic are delegated to service and DTOs.

Other diagrams

- Sequence diagram is appropriate for request flow from controller to service and repository.

Design for class “update-info.dto”



Package: server.modules.products.dtos

Role in diagram: Request DTO

Design note: Validates the fields allowed to update general product information.

Table 1. Attribute design for update-info.dto

#	Name	Data type	Default value	Description
1	title	string	""	Updated product title.
2	price	number	0	Updated selling price.

3	type	string	""	Updated product subtype/category.
---	------	--------	----	-----------------------------------

Table 2. Operation design for update-info.dto

#	Name	Return type	Description (purpose)
1	validate()	boolean	Checks that the update payload satisfies server rules.
2	toPartialEntity()	object	Converts validated data into a structure suitable for service update.

Parameter:

- Fields are supplied by the client update-info component.

Exception:

- Validation exception if field type, range or mandatory constraints are violated.

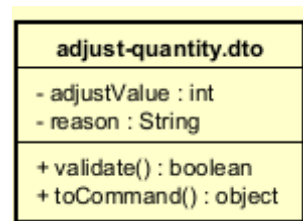
Method

- DTO is used by product.controller before delegating to product.service.

Other diagrams

- No extra diagram is necessary; validation rules may be listed in API documentation.

Design for class “adjust-quantity.dto”



Package: server.modules.products.dtos

Role in diagram: Request DTO

Design note: Carries the quantity change requested by the client quantity dialog.

Table 1. Attribute design for adjust-quantity.dto

#	Name	Data type	Default value	Description
1	adjustValue	number	0	Amount added to or subtracted from current stock.

2	reason	string	""	Optional explanation for the stock change to support history logging.
---	--------	--------	----	---

Table 2. Operation design for adjust-quantity.dto

#	Name	Return type	Description (purpose)
1	validate()	boolean	Ensures the adjustment is numerically valid.
2	toCommand()	object	Transforms the DTO into a service-layer command object.

Parameter:

- adjustValue: quantity delta requested by the user.
- reason: optional business explanation used in history logs.

Exception:

- Validation exception if the adjustment is not numeric or would violate server rules.

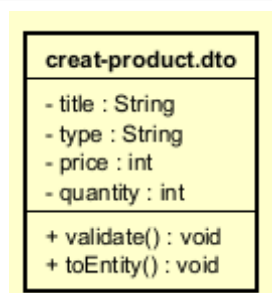
Method

- Used only in the quantity adjustment endpoint.

Other diagrams

- A small sequence diagram may be used if stock validation logic becomes complex.

Design for class “create-product.dto”



Package: server.modules.products.dtos

Role in diagram: Request DTO

Design note: Defines and validates the payload used to create a new product.

Table 1. Attribute design for create-product.dto

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	title	string	""	Name of the new product.
2	type	string	""	Subtype/category of the new product.
3	price	number	0	Initial selling price.
4	quantity	number	0	Initial stock quantity.

Table 2. Operation design for create-product.dto

#	Name	Return type	Description (purpose)
1	validate()	boolean	Checks completeness and basic business constraints.
2	toEntity()	Product	Builds a product entity or entity fragment for creation.

Parameter:

- Fields are provided by client product-form.

Exception:

- Validation exception if required fields are missing or invalid.

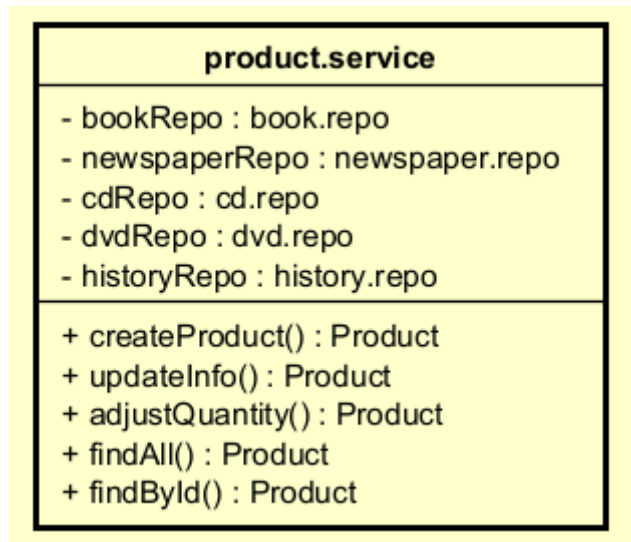
Method

- DTO shields the service layer from raw request payloads.

Other diagrams

- Activity or sequence diagram can be added for the create-product endpoint.

Design for class “product.service”



Package: server.modules.products.services

Role in diagram: Business service

Design note: Implements the main product use cases and coordinates repositories and logging.

Table 1. Attribute design for product.service

#	Name	Data type	Default value	Description
1	bookRepo	BookRepo	injected	Repository for book entities.
2	newspaperRepo	NewspaperRepo	injected	Repository for newspaper entities.
3	cdRepo	CdRepo	injected	Repository for CD entities.
4	dvdRepo	DvdRepo	injected	Repository for DVD entities.
5	historyRepo	HistoryRepo	injected	Repository used to store change history.

Table 2. Operation design for product.service

#	Name	Return type	Description (purpose)
1	createProduct(dto)	Product	Creates and stores a product of the requested subtype.
2	updateInfo(id, dto)	Product	Updates general product information.

3	adjustQuantity(id, dto)	Product	Changes stock quantity and records history.
4	findAll()	Product[]	Returns all products for list retrieval.
5	findById(id)	Product	Returns a product by identifier.

Parameter:

- id: target product id.
- dto: validated create/update/adjust payload.

Exception:

- Business exception if subtype-specific repository cannot find the target product or a rule is violated.

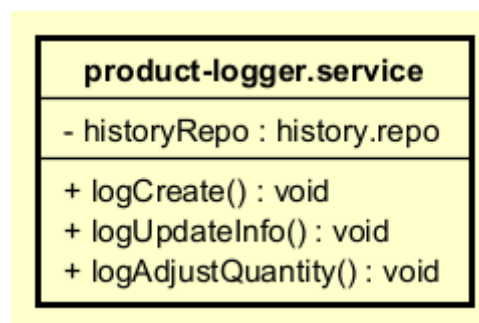
Method

- Central service in the server package.
- Chooses the correct repository according to product subtype and writes history logs when data changes.

Other diagrams

- Sequence diagram strongly recommended because this class coordinates controller, repositories and entities.

Design for class “product-logger.service”



Package: server.modules.products.services

Role in diagram: Logging service

Design note: Creates history records when product information or quantity changes.

Table 1. Attribute design for product-logger.service

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	historyRepo	HistoryRepo	injected	Repository used to persist history.entity instances.
---	-------------	-------------	----------	--

Table 2. Operation design for product-logger.service

#	Name	Return type	Description (purpose)
1	logCreate(product)	void	Stores a history record for product creation.
2	logUpdateInfo(product, dto)	void	Stores a history record for info updates.
3	logAdjustQuantity(product, dto)	void	Stores a history record for quantity changes.

Parameter:

- product: affected product entity.
- dto: payload that describes the performed change.

Exception:

- Persistence exception if history cannot be stored.

Method

- Designed as a supporting service to keep audit logic out of product.service.

Other diagrams

- Sequence diagram may be added if audit logging rules become more detailed.

Design for class “Payment-api.service”

payment-api.service	
- baseUrl : string - httpClient : HttpClient	
+ postRequest(endpoint : string, data : any) : void + getRequest(endpoint : string) : void + handleError(error : any) : void	

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	http	HttpClient		Angular's built-in HTTP client injected via Dependency Injection to perform external API calls.
2	baseUrl	String	'/api'	The base endpoint URL for the backend API, typically imported from the core constants (api-url) or environment configuration.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	postRequest(endpoint, payload)	Observable<any>	Sends an asynchronous HTTP POST request to the server with the payment payload.
2	getRequest(endpoint)	Observable<any>	Sends an asynchronous HTTP GET request to fetch data from the server.

3	handleError(error)	void	Fetches the latest status of a specific invoice from the backend. This is primarily used for polling when the user is paying via QR code, waiting for the bank transfer to register.
---	--------------------	------	--

Method: postRequest

- **Parameter:**
 - endpoint: String, the specific API URL path.
 - payload: Object, the data to be sent in the request body.
 - retryCount: Number (Default: 3), the number of times to retry if the request fails.
- **Exception:**
 - NetworkException if the server is unreachable or the connection times out.
 - AuthException if the API key or Bearer token is invalid/expired.
- **Method Logic:**
 - Attaches necessary security headers (Authorization, API-Key).
 - Serializes the payload into JSON format.
 - Uses the httpClient to execute the asynchronous POST call.

Method: handleError

- **Parameter:**
 - error: HttpResponseException, the raw error object returned by the server.
- **Exception:**
 - Throws specific custom exceptions (e.g., ServerError, ClientError) based on HTTP status codes.
- **Method Logic:**
 - Parses the error object to extract the error code and message.
 - Logs the technical details to the console for debugging.
 - Returns a user-friendly error message string to be displayed in the UI.

payment.service
- currentTransaction : string - config : PaymentConfig
+ initiatePayment(order : OrderModel) : Observable<any> + verifyTransaction(id : string) : Promise<boolean> + getPaymentMethods() : string[]

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	currentTransaction	Transaction	null	Dependency Injection (DI) automatically injected by NestJS to handle business logic and database interactions related to invoices.
2	config	PaymentConfig	null	Holds configuration settings and keys for the payment provider.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	initiatePayment	void	Starts the payment flow by sending order details to the backend and handling the subsequent UI routing or redirects based on the chosen gateway.
	getCurrentOrder	Observable<OrderModel>	Periodically checks the backend to see if a pending transaction (specifically VietQR) has been successfully paid by the user scanning the code on their banking app.
	clearPaymentState	void	Resets the frontend payment session data. This is crucial to prevent data leakage between different transactions after a payment succeeds or is canceled.

Method: initiatePayment

- **Parameter:**
 - order: OrderModel (Required), the data object containing order details and total amount.
 - method: String (Default: 'CreditCard'), the payment method selected by the user.
- **Exception:**
 - InvalidOrderException if the order object is null or the amount is less than or equal to zero.
 - UnsupportedMethodException if the selected payment method is not integrated into the system.
- **Method Logic:**
 - Validates the order data integrity.
 - Communicates with PaymentApiService to create a transaction on the server.
 - Maps the API response to the currentTransaction attribute.

Method: verifyTransaction

- **Parameter:**
 - transactionId: String (Required), the unique identifier of the transaction to be verified.
- **Exception:**
 - TransactionNotFoundException if the provided ID does not exist in the database.
- **Method Logic:**
 - Calls the backend endpoint via PaymentApiService to fetch the latest status.
 - Updates the local state based on whether the payment was 'Successful', 'Pending', or 'Failed'.

Design for class “Order.model”

order.model
- orderId : string - userId : string - amount : decimal - currency : string - status : string - createdAt : Date

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	String	N/A	Unique identifier for the order.
2	userId	String	N/A	Identifier of the user who created the order.
3	amount	Decimal	0	Total amount of the order.
4	currency	String	VND	Currency used for payment.
5	status	String	PENDING	Current status of the order.
6	createdAt	Date	current_date	Timestamp when the order was created.

Design for class “Invoice.model”

invoice.model

- invoiceId : string
- orderId : string
- issuedDate : Date
- taxAmount : decimal
- billingAddress : string

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	invoiceId	String	N/A	Unique identifier for the invoice.
2	orderId	String	N/A	Associated order identifier.
3	issuedDate	Date	current_date	Date when the invoice is issued.
4	taxAmount	Decimal	0	Tax amount applied.
5	billingAddress	String		Billing address.

Design for class “Qr.model”

qr.model

- qrCodeData : string
- expiryTime : int
- paymentUrl : string
- bankCode : string

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	qrCodeData	String	N/A	Encoded data inside QR.
2	expiryTime	Integer	N/A	Expiry time.
3	paymentUrl	String	N/A	Payment URL.
4	bankCode	String	N/A	Bank identifier.

Design for class “Refund.model”

refund.model

- refundId : string
- transactionId : string
- reason : string
- refundAmount : decimal
- status : string

Table 1. Attribute design

	Name	Data type	Default value	Description
1	refundId	String	N/A	Unique identifier.
2	transactionId	String	N/A	Related transaction.
3	reason	String		Reason for refund.
4	refundAmount	Decimal	0	Refund amount.
5	status	String	PROCESSING	Refund status.

3.2 PayOrder (servser)

Detailed Specification for Class Design

Design for class “PaymentService”

PaymentService
- vietqrService : VietqrService - paypalService : PaypalService
+ requestPayment(requestPaymentDto : RequestPaymentDto) : PaymentResultDto + changePaymentMethod(changePaymentMethodDto : ChangePaymentMethodDto) : PaymentResultDto + confirmTransaction(confirmTransactionDto : ConfirmTransactionDto) : PaymentResultDto + createPaymentTransaction(invoiceId : string, method : string, amount : number) : PaymentTransaction + updateTransactionSuccess(transactionId : string, externalTransactionId : string, content : string, transactionDateTime : Date) : PaymentResultDto + updateTransactionFailure(transactionId : string) : PaymentResultDto + handleRejectedOrderRefund(transactionId : string) : void + getPaymentTransactionByOrderId(orderId : string) : PaymentTransaction

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	vietqrService	VietqrService	null	Service used to generate QR code, handle callback, and sync transaction status for VietQR payments.

2	paypalService	PaypalService	null	Service used to create PayPal order, capture payment, and refund payment when needed.
---	---------------	---------------	------	---

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	requestPayment	PaymentResultDto	Create a payment request for an order using the selected payment method.
2	changePaymentMethod	PaymentResultDto	Change payment method from the current one to another supported method.
3	confirmTransaction	PaymentResultDto	Confirm payment transaction after receiving successful payment information.
4	createPaymentTransaction	PaymentTransaction	Create and store a payment transaction record for the order.
5	updateTransactionSuccess	PaymentResultDto	Update transaction as successful using returned transaction information.
6	updateTransactionFailure	PaymentResultDto	Update transaction as failed when payment is not completed or invalid.
7	handleRejectedOrderRefund	void	Process refund logic when an already paid order is later rejected.

8	getPaymentTransactionByOrderId	PaymentTransaction	Retrieve the payment transaction of a specific order.
---	--------------------------------	--------------------	---

Parameter:

- requestPaymentDto: payment request data including order id, invoice id, payment method, amount, and customer email.
- changePaymentMethodDto: data used to switch from one payment method to another.
- confirmTransactionDto: data used to confirm a completed transaction, including transaction id, content, datetime, and status.
- invoiceId: id of the invoice associated with the payment.
- method: selected payment method, such as VietQR or PayPal.
- amount: total amount to be paid.
- transactionId: internal payment transaction id.
- externalTransactionId: transaction id returned by external payment gateway.
- content: transaction content returned after successful payment.
- transactionDateTime: date and time when the payment transaction was completed.
- orderId: id of the order whose payment transaction is queried.

Exception:

- PaymentMethodNotSupportedException if the selected payment method is not supported by the system.
- TransactionNotFoundException if the payment transaction does not exist.
- InvalidTransactionStatusException if the transaction status is invalid for confirmation or update.
- RefundNotSupportedException if refund is requested for a payment method that does not support automatic refund.

Method

- requestPayment() is used when the customer proceeds to payment after invoice information has been prepared.

-
- `changePaymentMethod()` is used when the customer wants to switch from default QR code payment to PayPal or another supported method.
 - `confirmTransaction()` is used after successful payment to record payment transaction information.
 - `createPaymentTransaction()` is used before sending the payment request to an external gateway.
 - `updateTransactionSuccess()` stores successful transaction data such as transaction id, transaction content, and transaction datetime.
 - `updateTransactionFailure()` marks the transaction as failed.
 - `handleRejectedOrderRefund()` handles refund after a paid order is rejected. For PayPal this can be automated, while VietQR requires manual refund handling.
 - `getPaymentTransactionByOrderId()` is used to retrieve the stored payment transaction for later processing or display.

Other diagrams

Design for class “PaymentTransaction”

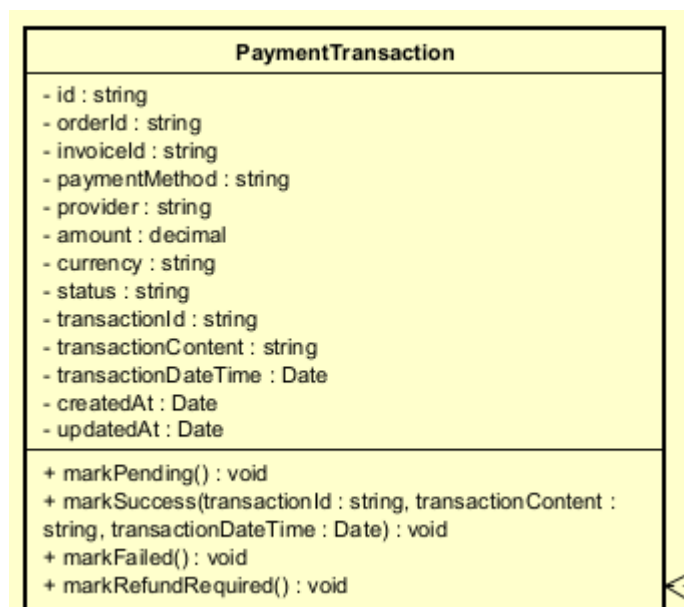


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	id	string	"	Unique identifier of the payment transaction.
2	orderId	string	"	Identifier of the order associated with this payment transaction.
3	invoiceId	string	"	Identifier of the invoice associated with this payment transaction.
4	paymentMethod	string	'VIETQR'	Selected payment method used by the customer.
5	provider	string	"	External payment provider, such as vietqr or paypal.
6	amount	number	0	Total amount of the transaction.
7	status	string	'pending'	Current status of the payment transaction.
8	transactionId	string	"	External transaction identifier returned by the payment gateway.
9	transactionContent	string	"	Transaction content or description returned after payment.
10	transactionDateTime	Date	null	Date and time when the payment transaction was completed.

11	createdAt	Date	new Date()	Timestamp when the payment transaction record was created.
12	updatedAt	Date	new Date()	Timestamp when the payment transaction record was last updated.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	markPending	void	Set the transaction status to pending.
2	markSuccess	void	Set the transaction status to success and store returned gateway information.
3	markFailed	void	Set the transaction status to failed.
4	markRefundRequired	void	Set the transaction status to indicate manual or automatic refund handling is required.

Parameter:

- transactionId: external transaction id returned by the payment gateway.
- transactionContent: content associated with the transaction.
- transactionDateTime: date and time when the payment was completed.

Exception:

- InvalidStatusTransitionException if the transaction is updated to an invalid status.

Method

- markPending() is used immediately after creating the transaction record.

-
- markSuccess() is used after a successful payment callback or capture result is received.
 - markFailed() is used when the payment attempt fails or is rejected.
 - markRefundRequired() is used when the order is rejected after successful payment and refund handling is needed.

Other diagrams

Design for class “PaymentController”

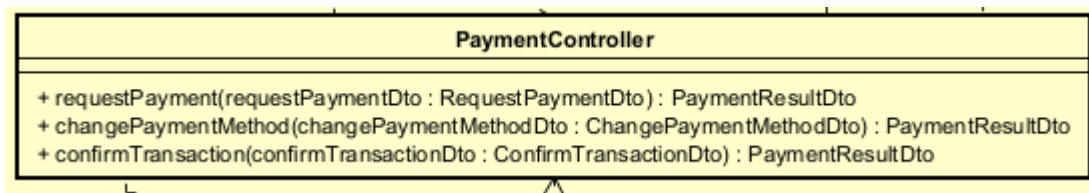


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	requestPayment	PaymentResultDto	Receive payment request data from the client and forward it to PaymentService.
2	changePaymentMethod	PaymentResultDto	Receive request to change payment method and forward it to PaymentService.

3	confirmTransaction	PaymentResultDto	Receive transaction confirmation data and forward it to PaymentService.
---	--------------------	------------------	---

Parameter:

- requestPaymentDto: data required to start a payment request.
- changePaymentMethodDto: data required to switch payment method.
- confirmTransactionDto: data required to confirm transaction result.

Exception:

- BadRequestException if request data is missing or invalid.
- PaymentMethodNotSupportedException if the requested payment method is not supported.

Method

- requestPayment() is called by the client when the customer proceeds to payment.
- changePaymentMethod() is called when the customer changes from one supported payment method to another.
- confirmTransaction() is called after receiving successful payment information that must be confirmed and saved.

Other diagrams

Design for class “RequestPaymentDto”

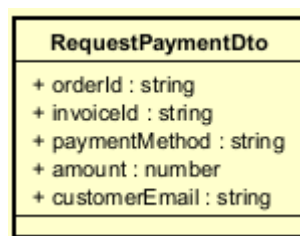


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	orderId	string	"	Identifier of the order to be paid.
2	invoiceId	string	"	Identifier of the invoice associated with the order.
3	paymentMethod	string	'VIETQR'	Selected payment method.
4	amount	number	0	Total amount to be paid.
5	customerEmail	string	"	Email address used for payment or notification purposes.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries data required to create a payment request.

Other diagrams

Design for class “ChangePaymentMethodDto”

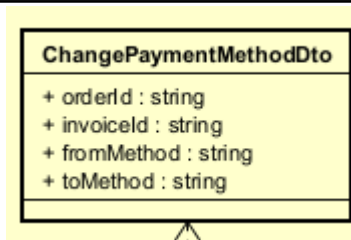


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	orderId	string	"	Identifier of the related order.
2	invoiceId	string	"	Identifier of the related invoice.
3	fromMethod	string	"	Current payment method.
4	toMethod	string	"	Target payment method to switch to.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries data required to switch from one payment method to another.

Other diagrams

Design for class “ConfirmTransactionDto”

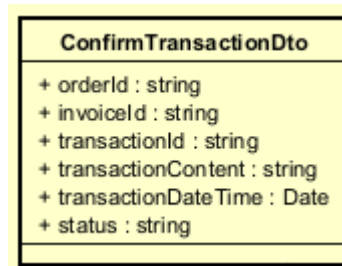


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	orderId	string	"	Identifier of the related order.
2	invoiceId	string	"	Identifier of the related invoice.
3	transactionId	string	"	Transaction id returned by the payment gateway.
4	transactionContent	string	"	Transaction content returned by the payment gateway.
5	transactionDateTime	Date	null	Date and time when the transaction was completed.
6	status	string	'success'	Status of the confirmed transaction.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries data required to confirm a completed transaction.

Other diagrams

Design for class “PaymentResultDto”

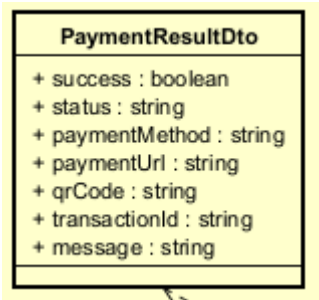


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	success	boolean	false	Indicates whether the payment-related action is successful.
2	status	string	"	Current payment or transaction status.

3	paymentMethod	string	"	Payment method used for the current payment flow.
4	paymentUrl	string	"	Redirect URL returned by PayPal if applicable.
5	qrCode	string	"	QR code content returned for VietQR payment if applicable.
6	transactionId	string	"	Transaction id returned by the payment gateway if available.
7	message	string	"	Message describing the payment result.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries the response data of a payment-related action back to the client.

Other diagrams

Design for class “PaypalController”

PayPalController
+ createOrder(createOrderDto : CreateOrderDto) : PaymentResultDto + capturePayment(capturePaymentDto : CapturePaymentDto) : PaymentResultDto + refundPayment(refundDto : RefundDto) : PaymentResultDto

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	createOrder	PaymentResultDto	Receive PayPal order creation data and forward it to PaypalService.
2	capturePayment	PaymentResultDto	Receive capture request and finalize a PayPal payment.
3	refundPayment	PaymentResultDto	Receive refund request data and forward it to PaypalService.

Parameter:

- createOrderDto: data required to create a PayPal order.
- capturePaymentDto: data required to capture an approved PayPal order.
- refundDto: data required to refund a captured PayPal payment.

Exception:

- BadRequestException if request data is missing or invalid.
- PaypalApiException if communication with PayPal fails.

Method

- createOrder() is called to create a PayPal order when the customer selects PayPal.
- capturePayment() is called after customer approval to complete the payment.
- refundPayment() is called when a successful PayPal payment must be refunded.

Other diagrams

Design for class “PaypalService”

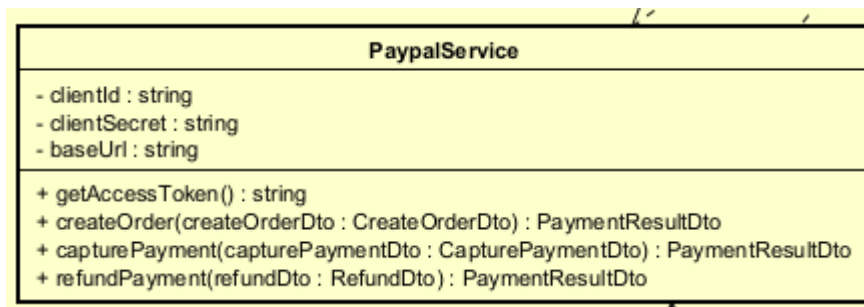


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	clientId	string	"	PayPal client id used for authentication.
2	clientSecret	string	"	PayPal client secret used for authentication.
3	baseUrl	string	"	Base URL of PayPal sandbox API.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	getAccessToken	string	Obtain PayPal access token for calling PayPal APIs.
2	createOrder	PaymentResultDto	Create a PayPal order for the given payment request.
3	capturePayment	PaymentResultDto	Capture an approved PayPal payment.
4	refundPayment	PaymentResultDto	Refund a completed PayPal payment.

Parameter:

- createOrderDto: order creation data including amount, currency, and order reference.
- capturePaymentDto: data required to capture an approved PayPal order.
- refundDto: data required to refund a captured PayPal transaction.

Exception:

- AuthenticationException if access token cannot be retrieved.
- PaypalApiException if PayPal returns an error response.

Method

- getAccessToken() is used before calling protected PayPal APIs.
- createOrder() creates a PayPal order and returns payment-related result data.
- capturePayment() finalizes a PayPal payment after user approval.
- refundPayment() issues a refund for a completed PayPal transaction.

Other diagrams**Design for class “CreateOrderDto”**

CreateOrderDto
+ orderId : string
+ invoiceId : string
+ amount : number
+ currency : string
+ description : string

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	orderId	string	"	Identifier of the related order.
2	invoiceId	string	"	Identifier of the related invoice.
3	amount	number	0	Amount to be charged through PayPal.
4	currency	string	'USD'	Currency code used for the PayPal order.
5	description	string	"	Short description of the order or payment.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries data required to create a PayPal order.

Other diagrams

Design for class “CapturePaymentDto”

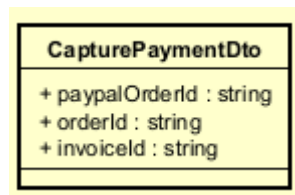


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	paypalOrderId	string	"	PayPal order id returned after order creation.
2	orderId	string	"	Identifier of the related order.
3	invoiceId	string	"	Identifier of the related invoice.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries data required to capture an approved PayPal payment.

Other diagrams

Design for class “RefundDto”

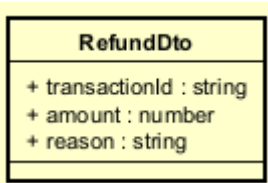


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	transactionId	string	"	Identifier of the completed PayPal transaction to refund.
2	amount	number	0	Refund amount.
3	reason	string	"	Reason for refund.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries data required to refund a PayPal payment.

Other diagrams

Design for class “VietqrController”

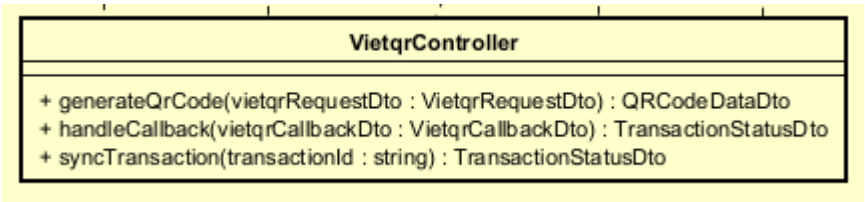


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	generateQrCode	QRCodeDataDto	Receive VietQR payment request data and return generated QR code data.

2	handleCallback	TransactionStatusDto	Receive callback data from VietQR and return normalized transaction status data.
3	syncTransaction	TransactionStatusDto	Query or synchronize transaction status for a specific transaction.

Parameter:

- vietqrRequestDto: data required to create a VietQR payment QR code.
- vietqrCallbackDto: callback data returned by VietQR after payment processing.
- transactionId: identifier of the transaction to synchronize or query.

Exception:

- BadRequestException if request data is missing or invalid.
- VietqrApiException if communication with VietQR fails.

Method

- generateQrCode() is called when the customer chooses or is assigned VietQR payment.
- handleCallback() is called by VietQR callback endpoint after payment status changes.
- syncTransaction() is used to query or synchronize transaction status when needed.

Other diagrams

Design for class “VietqrService”

VietqrService	
- clientId : string	
- apiKey : string	
- accessToken : string	
- baseUrl : string	
+ getAccessToken() : string	
+ generateQrCode(vietqrRequestDto : VietqrRequestDto) : QRCodeDataDto	
+ handleCallback(vietqrCallbackDto : VietqrCallbackDto) : TransactionStatusDto	
+ syncTransaction(transactionId : string) : TransactionStatusDto	
+ mapCallbackToTransactionStatus(vietqrCallbackDto : VietqrCallbackDto) : TransactionStatusDto	

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	clientId	string	"	VietQR client id used for authentication or integration.
2	apiKey	string	"	API key used to access VietQR services.
3	accessToken	string	"	Temporary access token used to call VietQR APIs.
4	baseUrl	string	"	Base URL of VietQR sandbox API.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	getAccessToken	string	Obtain VietQR access token for calling protected APIs.
2	generateQrCode	QRCodeDataDto	Generate QR code data for a VietQR payment request.

3	handleCallback	TransactionStatusDto	Process callback data returned by VietQR.
4	syncTransaction	TransactionStatusDto	Synchronize transaction status from VietQR.
5	mapCallbackToTransactionStatus	TransactionStatusDto	Convert VietQR callback data into normalized transaction status data.

Parameter:

- vietqrRequestDto: data required to create a VietQR payment QR code.
- vietqrCallbackDto: callback data returned by VietQR after payment processing.
- transactionId: identifier of the transaction to synchronize or query.

Exception:

- AuthenticationException if access token cannot be retrieved.
- VietqrApiException if VietQR returns an error response.

Method

- getAccessToken() is used before calling protected VietQR APIs.
- generateQrCode() creates QR code data that the client can display to the customer.
- handleCallback() processes callback payload received from VietQR.
- syncTransaction() retrieves or synchronizes latest transaction status from VietQR.
- mapCallbackToTransactionStatus() converts raw callback data to a normalized status object.

Other diagrams

Design for class “VietqrRequestDto”

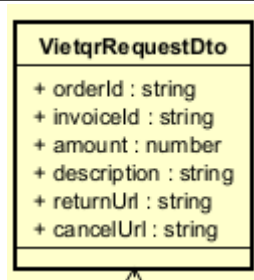


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	orderId	string	"	Identifier of the related order.
2	invoiceId	string	"	Identifier of the related invoice.
3	amount	number	0	Amount to be paid through VietQR.
4	description	string	"	Short description or transaction content for the payment.
5	returnUrl	string	"	Return URL used after payment if applicable.
6	cancelUrl	string	"	Cancel URL used when payment is canceled if applicable.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries data required to generate a VietQR code.

Other diagrams

Design for class “QRCodeDataDto”

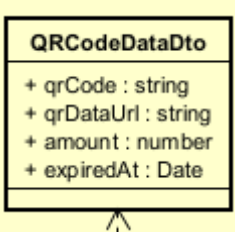


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	qrCode	string	"	Encoded QR code content.
2	qrDataUrl	string	"	Data URL or image URL of generated QR code.
3	amount	number	0	Amount represented by the QR code.
4	expiredAt	Date	null	Expiration time of the QR code if applicable.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries VietQR QR code data returned to the client.

Other diagrams

Design for class “VietqrCallbackDto”

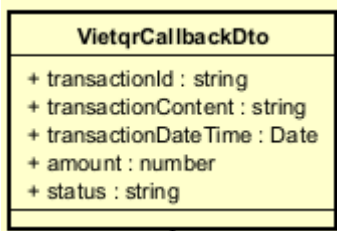


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	transactionId	string	"	Identifier of the VietQR transaction.
2	transactionContent	string	"	Transaction content returned in callback data.

3	transactionDateTime	Date	null	Date and time when the callback transaction was completed.
4	amount	number	0	Paid amount returned in callback data.
5	status	string	"	Transaction status returned in callback data.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries callback data returned by VietQR.

Other diagrams

Design for class “TransactionStatusDto”

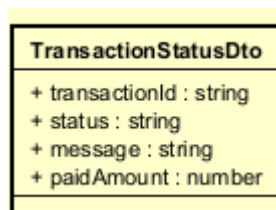


Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	transactionId	string	"	Identifier of the related transaction.
2	status	string	"	Normalized status of the transaction.
3	message	string	"	Message describing the current transaction state.
4	paidAmount	number	0	Amount confirmed as paid.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

Parameter:

Exception:

Method

- This DTO carries normalized transaction status data.

Other diagrams

Design for class “ProductListComponent”

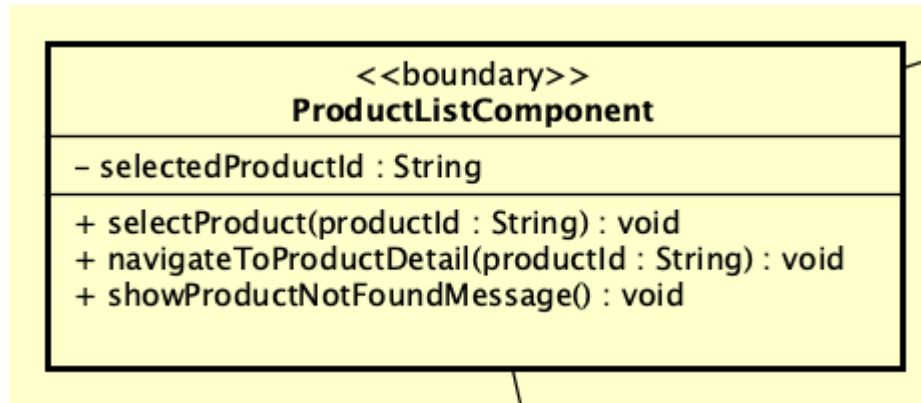


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	selectedProductId	String		Stores the identifier of the product selected by the Customer from the product list.
2	productApiService	ProductAPIService	injected	Client-side service used to request product detail data from the server.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	selectProduct(productId)	void	Captures the user's selection of a product from the list.
2	navigateToProductDetail(productId)	void	Navigates the user to the product detail view for the selected product.
3	showProductNotFoundMessage()	void	Displays a message when the selected product does not exist.

Parameter:

- **productId:** String; default value: N/A; Identifier of the selected product.

Exception:

- No business exception is thrown directly by this boundary class; display errors are propagated from ProductContainerComponent or ProductAPIService.

Method

- selectProduct(productId) saves the selected product identifier and delegates navigation to navigateToProductDetail(productId). This class supports the first user interaction of UC235, where the Customer selects a product from a product list, homepage, or search result.

Other diagrams

- Sequence diagram of UC235
- Communication diagram of UC235

Design for class “ProductManagementComponent”

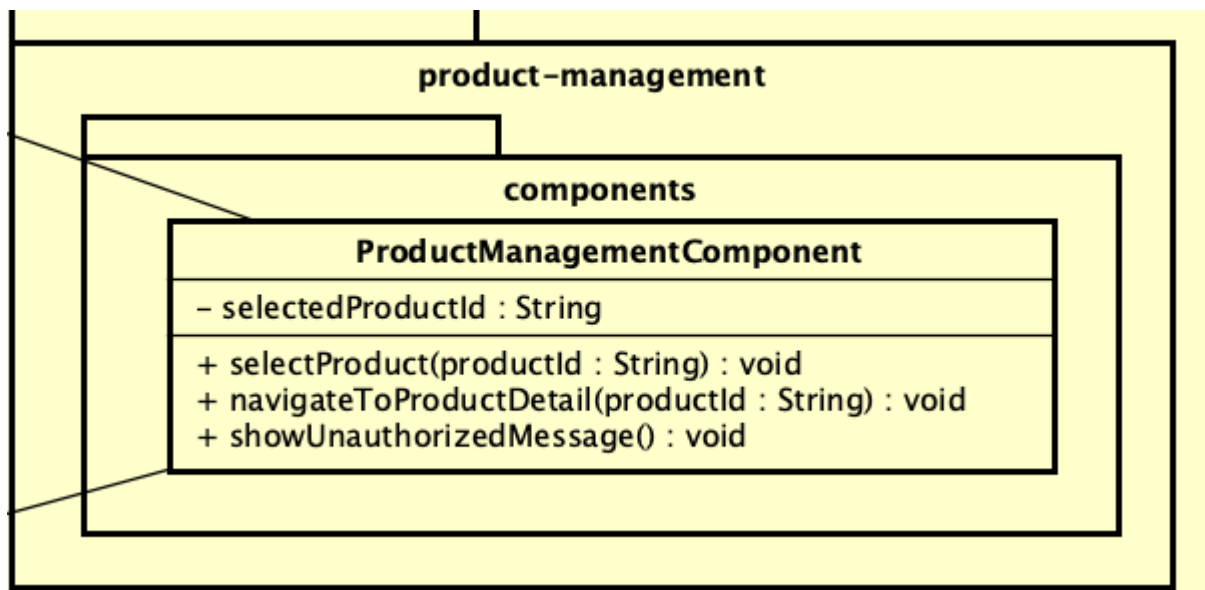


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	selectedProductId	String		Stores the identifier of the product selected by the Product Manager.

2	productApiService	ProductAPIService	injected	Client-side service used to request detailed product information.
---	-------------------	-------------------	----------	---

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	selectProduct(productId)	void	Captures the Product Manager's selection of a product.
2	navigateToProductDetail(productId)	void	Navigates to the product detail view.
3	showUnauthorizedMessage()	void	Displays an authorization warning when the Product Manager is not allowed to access the detail view.

Parameter:

- **productId:** String; default value: N/A; Identifier of the selected product.

Exception:

- No direct exception is raised in this component; unauthorized results are displayed as UI feedback.

Method

- This class is the Product Manager entry boundary for the use case. After a selection is made, it triggers the navigation to the detail view and presents an authorization message if the server rejects the request.

Other diagrams

- Sequence diagram of UC235

Design for class “ProductContainerComponent”

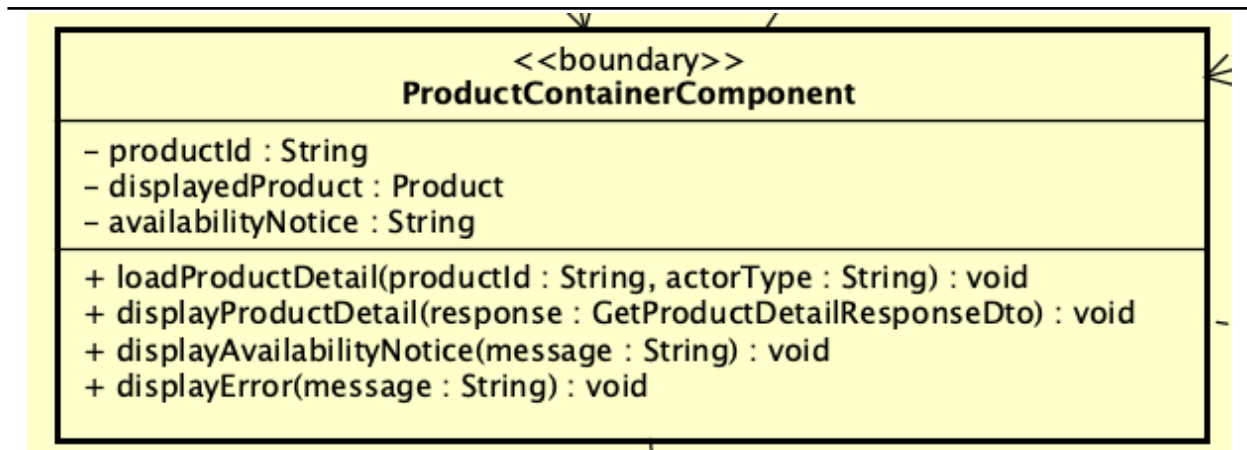


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	String		Identifier of the product currently displayed.
2	displayedProduct	GetProductDetailResponse Dto	null	Response object containing all data required for rendering the product detail page.
3	availabilityNotice	String		Availability message shown to Customer when the product is unavailable for sale.
4	productApiService	ProductAPIService	injected	Service used to call the backend API for product detail retrieval.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	loadProductDetail(productId, actorType)	void	Requests detailed information for the selected product according to the current actor type.

2	displayProductDetail(response)	void	Renders the returned product detail information on the UI.
3	displayAvailabilityNotice(message)	void	Displays an availability notice before or together with the detail page.
4	displayError(message)	void	Displays an error message such as not found, unauthorized, or load failure.

Parameter:

- **productId:** String; default value: N/A; Identifier of the requested product.
- **actorType:** String; default value: 'CUSTOMER'; Current actor role, for example CUSTOMER or PRODUCT_MANAGER.
- **response:** GetProductDetailResponseDto; default value: N/A; Returned detail data to be rendered.
- **message:** String; default value: N/A; Error message or availability notice.

Exception:

- **NotFoundException** if the selected product does not exist.
- **UnauthorizedException** if the Product Manager is not authenticated or authorized.
- **DataAccessException** if product information or media cannot be retrieved.

Method

- loadProductDetail(productId, actorType) creates a request DTO and invokes ProductAPIService.getProductDetail(request). If the response is successful, displayProductDetail(response) is called. If the response contains an availability notice for a Customer, displayAvailabilityNotice(message) is called before or together with rendering. If an error occurs, displayError(message) is used to show feedback on the UI.

Other diagrams

- Sequence diagram of UC235
- Activity diagram of UC235

Design for class “ProductAPIService”

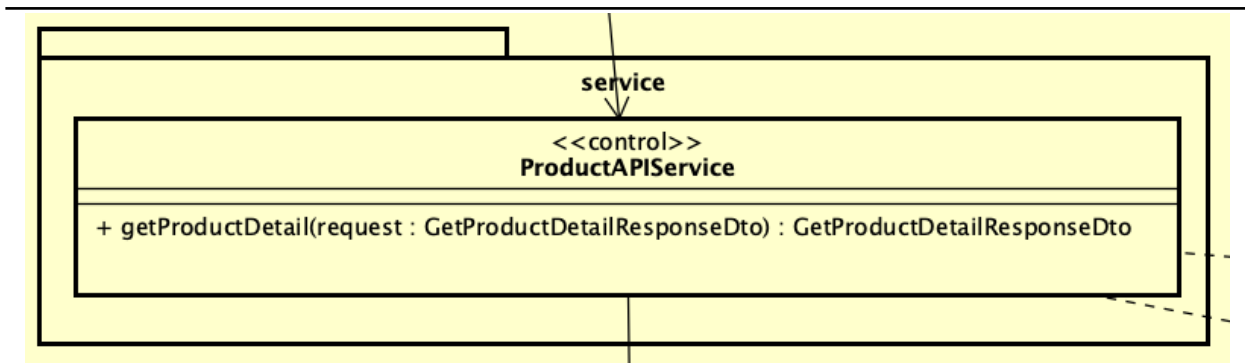


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	baseUrl	String	"/api/products"	Base endpoint used by the frontend to access product-related backend APIs.
2	httpClient	HttpClient	injected	HTTP client used to send requests from frontend to backend.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getProductDetail(request)	GetProductDetailResponse Dto	Sends a request to the backend and returns the product detail response DTO.

Parameter:

- **request:** GetProductDetailRequestDto; default value: N/A; Request object containing productId and actorType.

Exception:

- **HttpException** if the server responds with an HTTP error status.
- **TimeoutException** if the request exceeds the configured response timeout.

Method

- This class serializes the request DTO, sends it to ProductController through HTTP, receives the response payload, and deserializes it into GetProductDetailResponseDto for ProductContainerComponent.

Other diagrams

- Sequence diagram FE-BE interaction

Design for class “GetProductDetailRequestDto”

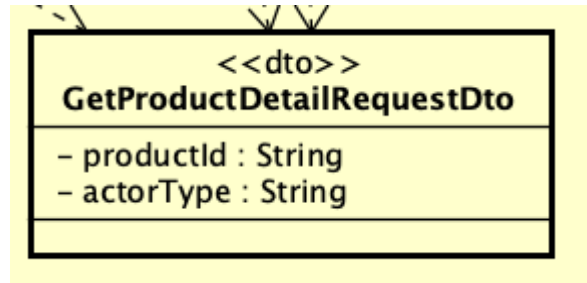


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	String		Identifier of the product to be viewed.
2	actorType	String	'CUSTOMER'	Type of actor making the request, such as CUSTOMER or PRODUCT_MANAGER.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	validate()	boolean	Validates the minimal integrity of the request data.

Parameter:

- None

Exception:

- **ValidationException** if productId is blank or actorType is invalid.

Method

- This DTO is a simple data carrier. validate() checks that productId is present and actorType belongs to the supported actor set.

Other diagrams

- No additional diagram is required.

Design for class “GetProductDetailResponseDto”

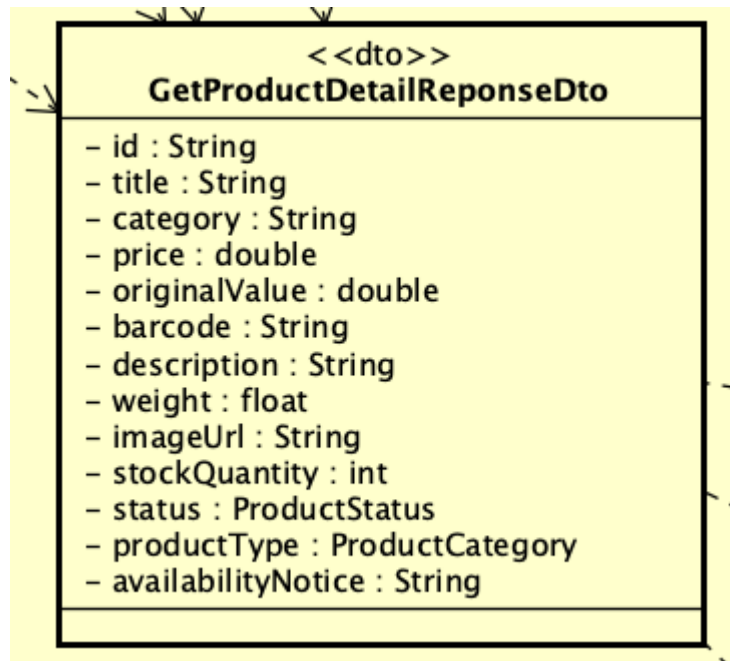


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	id	String		Unique identifier of the product.
2	title	String		Product title displayed as the main heading.
3	category	ProductCategory	null	Type/category of the product.
4	price	double	0.0	Current selling price.
5	originalValue	double	0.0	Original value of the product.
6	barcode	String		Barcode identifying the product.
7	description	String		General description of the product.
8	weight	float	0.0	Physical weight of the product.

9	imageUrl	String		URL or path to the main image or thumbnail.
10	stockQuantity	int	0	Current quantity in stock.
11	status	ProductStatus	null	Current status of the product.
12	availabilityNotice	String		Notice shown to Customer when the product is unavailable.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	fromProduct(product, actorType)	GetProductDetailResponse Dto	Builds a response DTO from a product entity and the current actor context.

Parameter:

- **product:** Product; default value: N/A; Source product entity.
- **actorType:** String; default value: 'CUSTOMER'; Current actor role used to shape the response content.

Exception:

- **MappingException** if the product entity lacks required information for response construction.

Method

- This DTO aggregates the general product information and status needed by the UI. The builder/mapping operation converts a Product entity into a transport-friendly structure that the frontend can render directly.

Other diagrams

- No additional diagram is required.

Design for class “ProductController”

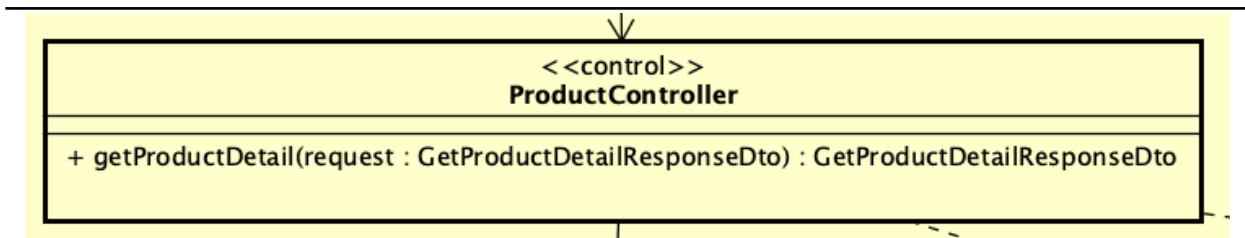


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productService	ProductService	injected	Application service that handles the product detail retrieval use case.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getProductDetail(request)	GetProductDetailResponse Dto	Accepts the client request and returns the detailed product response.

Parameter:

- **request:** GetProductDetailRequestDto; default value: N/A; Request object containing the selected product identifier and actor type.

Exception:

- **BadRequestException** if the request DTO is invalid.
- **NotFoundException** if the target product does not exist.
- **UnauthorizedException** if the Product Manager is not permitted to access the detail.

Method

- The controller validates the request at boundary level, delegates the business logic to `ProductService.getProductDetail(request)`, and returns the resulting response DTO to the client.

Other diagrams

- Sequence diagram FE-BE interaction

Design for class “ProductService”

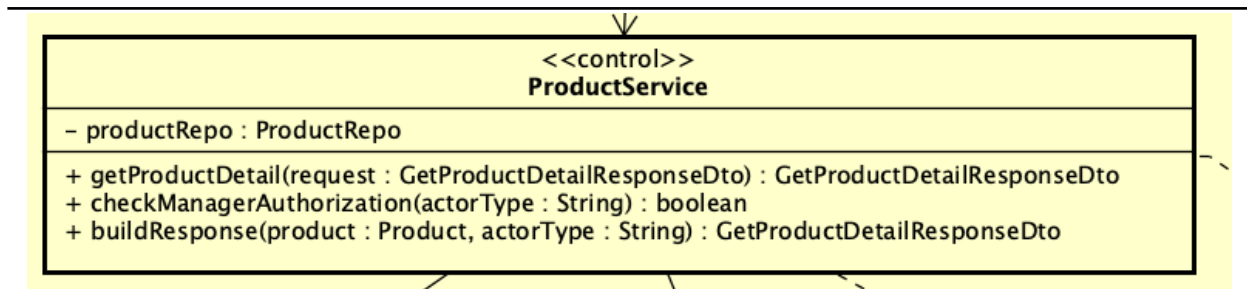


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productRepo	ProductRepo	injected	Repository used to retrieve product data from persistence.
2	accessGuard	ProductManagerAccessGuard	injected	Guard used to check whether a Product Manager is allowed to access the requested detail view.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getProductDetail(request)	GetProductDetailResponseDto	Coordinates the full use case of viewing detailed product information.
2	checkManagerAuthorization(actorType)	boolean	Determines whether the actor is allowed to access manager-level product detail.

3	buildResponse(product, actorType)	GetProductDetailResponse Dto	Constructs the final response object based on product data and actor context.
---	-----------------------------------	------------------------------	---

Parameter:

- **request:** GetProductDetailRequestDto; default value: N/A; Request coming from ProductController.
- **actorType:** String; default value: 'CUSTOMER'; Role of the actor requesting the detail.
- **product:** Product; default value: N/A; Product entity retrieved from the repository.

Exception:

- **NotFoundException** if ProductRepo.findById(productId) returns null.
- **UnauthorizedException** if the actor is a Product Manager but does not pass authorization.
- **DataAccessException** if persistence or related media retrieval fails.

Method

- getProductDetail(request) first extracts productId and actorType from the request. It asks ProductRepo to find the product. If no product is found, NotFoundException is raised. If the actor is Product Manager, checkManagerAuthorization(actorType) is called and an UnauthorizedException may be raised. Then the service evaluates the product status. For Customer, if the product is unavailable for sale, an availability notice is inserted into the response. Finally, buildResponse(product, actorType) returns GetProductDetailResponseDto.

Other diagrams

- Sequence diagram of UC235
- Activity diagram of UC235

Design for class “ProductRepo”

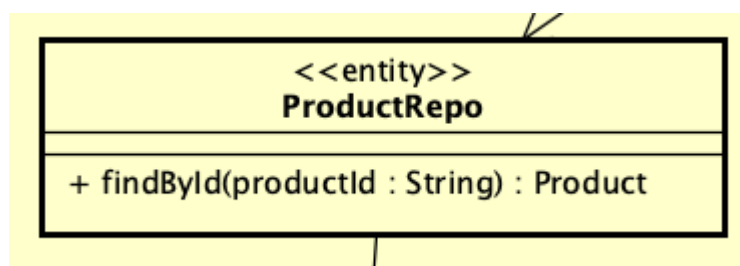


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	dataSource	DataSource	injected	Persistence access object or connection used to query product data.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	findById(productId)	Product	Retrieves a product and its related detail information by identifier.

Parameter:

- **productId:** String; default value: N/A; Identifier of the product to retrieve.

Exception:

- **DataAccessException** if the query fails.
- **ServiceException** if related media or associated detail records cannot be loaded.

Method

- This repository queries the product catalog and returns a fully constructed Product subtype, including common fields and type-specific associated data such as authors, sections, or tracks as needed by the selected product type.

Other diagrams

- Sequence diagram of backend interaction

Design for class “Product”

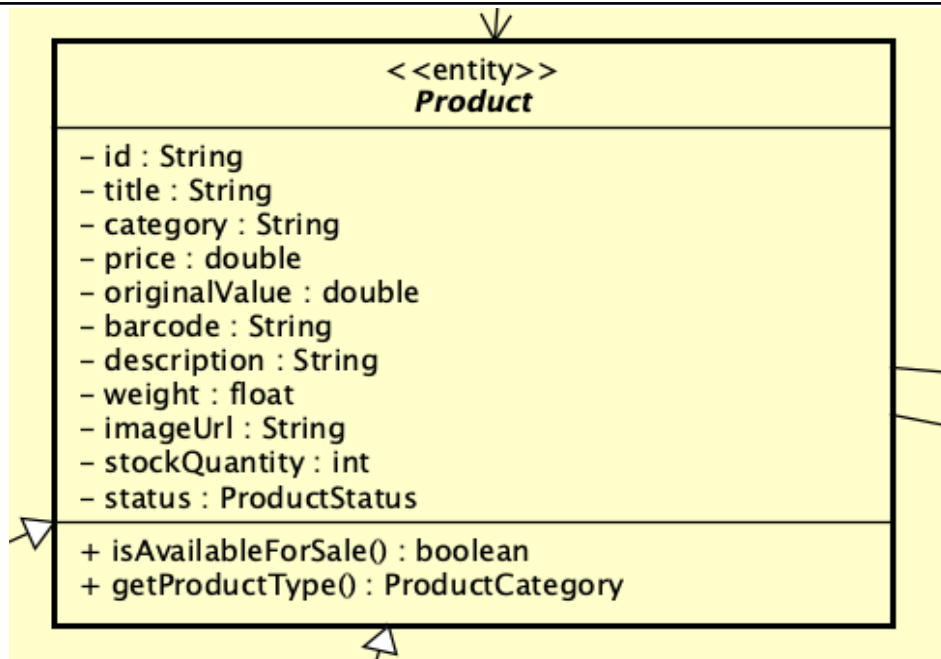


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	id	String		Unique product identifier.
2	title	String		Official title of the product.
3	category	ProductCategory	null	Product category or type.
4	price	double	0.0	Current selling price.
5	originalValue	double	0.0	Original value of the product.
6	barcode	String		Barcode used to identify the product.
7	description	String		General product description.
8	weight	float	0.0	Physical weight of the product.
9	imageUrl	String		Image path or URL of the product.
10	stockQuantity	int	0	Quantity currently available in stock.

11	status	ProductStatus	ACTIVE	Current state of the product in the catalog.
----	--------	---------------	--------	--

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	isAvailableForSale()	boolean	Checks whether the product can currently be sold to Customer.
2	getProductType()	ProductCategory	Returns the product category/type.

Parameter:

- None

Exception:

- No direct business exception is defined for this entity operation set.

Method

- isAvailableForSale() evaluates the status and stock quantity to decide whether the product is available for sale. This entity captures the common product attributes required by the AIMS problem statement and the View Product Detail use case.

Other diagrams

- No additional diagram is required.

Design for class “Book”

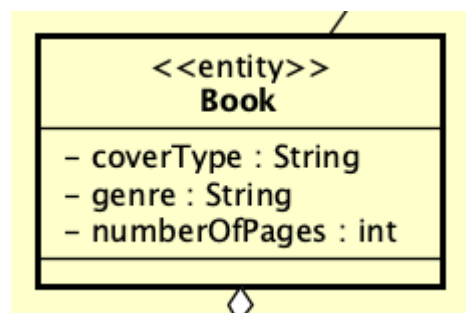


Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	coverType	String		Physical cover style of the book, such as paperback or hardcover.
2	genre	String		Literary genre of the book.
3	numberOfPages	int	0	Number of pages in the book.
4	authors	List<Author>	[]	List of authors associated with the book.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getAuthorNames()	String	Returns a display-friendly list of author names.

Parameter:

- None

Exception:

- No direct business exception is defined for this entity operation set.

Method

- This class extends PrintableProduct and stores the specific detail fields required for books in the product catalog and detail page.

Other diagrams

- No additional diagram is required.

Design for class “Newspaper”

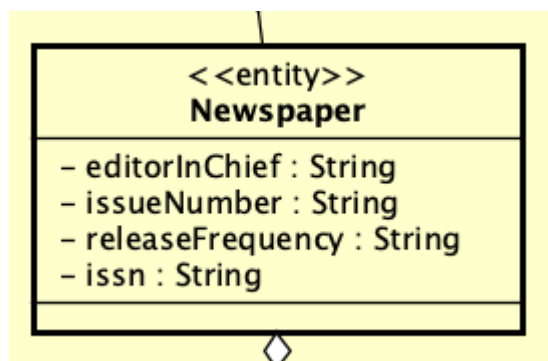


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	editorInChief	String		Editor-in-chief responsible for the newspaper issue.
2	issueNumber	String		Edition or issue number.
3	releaseFrequency	String		Publication frequency, for example daily or weekly.
4	issn	String		International Standard Serial Number.
5	sections	List<Section>	[]	Main content sections covered by the newspaper.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getSectionNames()	String	Returns a display-friendly list of section names.

Parameter:

- None

Exception:

- No direct business exception is defined for this entity operation set.

Method

- This class extends PrintableProduct and stores the specific detail fields required for newspapers in the product catalog and detail page.

Other diagrams

- No additional diagram is required.

Design for class “CD”

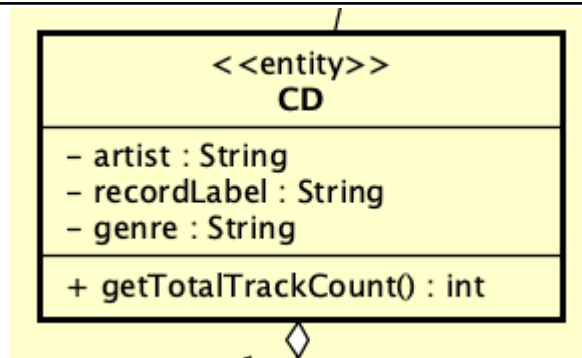


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	artist	String		Artist or band featured on the CD.
2	recordLabel	String		Label releasing the CD.
3	genre	String		Music genre.
4	tracks	List<Track>	[]	Track list contained in the CD.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getTotalTrackCount()	int	Returns the total number of tracks in the CD.

Parameter:

- None

Exception:

- No direct business exception is defined for this entity operation set.

Method

- This class extends DiscProduct and stores the detail fields required for CDs, including track data used on the product detail page.

Other diagrams

- No additional diagram is required.

Design for class “DVD”

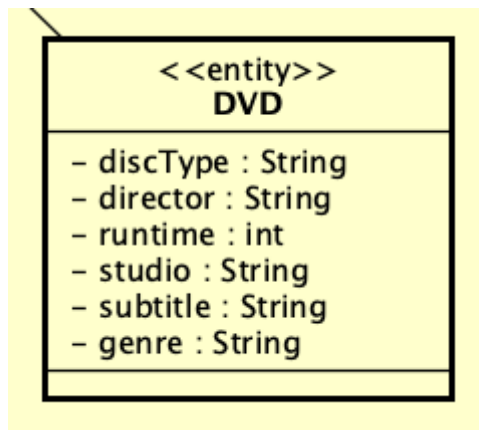


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	discType	String		Disc format, such as Blu-ray or HD-DVD.
2	director	String		Director of the video content.
3	runtime	int	0	Total running time in minutes.
4	studio	String		Production or distribution studio.
5	subtitle	String		Available subtitle information.
6	genre	String		Film genre of the DVD content.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getDisplaySubtitle()	String	Returns subtitle information in a display-friendly format.

Parameter:

- None

Exception:

- No direct business exception is defined for this entity operation set.

Method

- This class extends DiscProduct and stores the specific detail fields required for DVDs in the product catalog and detail page.

Other diagrams

- No additional diagram is required.

Design for class “Cart.controller”

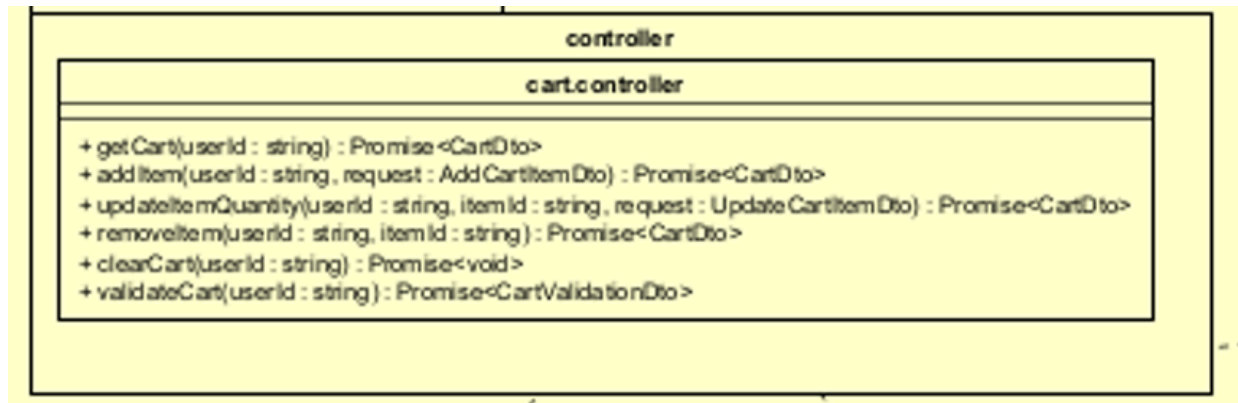


Table 2. Operation design

#	Name	Return type	Description (purpose)
1	+ getCart(userId)	Promise<CartDto>	Get the current cart of a given user and return a serialized cart DTO for the presentation/API layer.
2	+ addItem(userId, request)	Promise<CartDto>	Add a new product line to the user cart or increase quantity when the same product already exists.
3	+ updateItemQuantity(userId, itemId, request)	Promise<CartDto>	Update the quantity of an existing cart line after validating business constraints.
4	+ removeItem(userId, itemId)	Promise<CartDto>	Remove a specific line item from the current cart and return the updated cart state.
5	+ clearCart(userId)	Promise<void>	Delete all line items in the cart and reset the cart to an empty state.
6	+ validateCart(userId)	Promise<CartValidationDto>	Check whether the current cart is ready for the next step, including quantity, existence, and stock-related rules.

Parameter:

- `userId` (string): unique identifier of the cart owner; required in every controller operation.
- `request: AddCartItemDto`: carries `productId` and `quantity` for add-item requests.
- `itemId` (string): identifier of the target cart line used for update/remove operations.
- `request: UpdateCartItemDto`: carries the new quantity for quantity-update requests.

Exception:

- `CartNotFoundException` if the requested cart does not exist.
- `CartItemNotFoundException` if the requested line item is absent.
- `InvalidCartItemQuantityException` if quantity is null, zero, negative, or otherwise invalid.
- `EmptyCartException` if validation is requested on an empty cart.
- `ProductOutOfStockException` if one or more items cannot be fulfilled from stock.

Method

The controller owns no business state. Each public operation validates inbound parameters at API level, delegates the use case to `cartService`, and returns a DTO suitable for the boundary layer. Exception mapping should stay centralized so HTTP/API responses remain consistent.

Other diagrams

- No additional diagram is required.

Design for class “Cart.service”

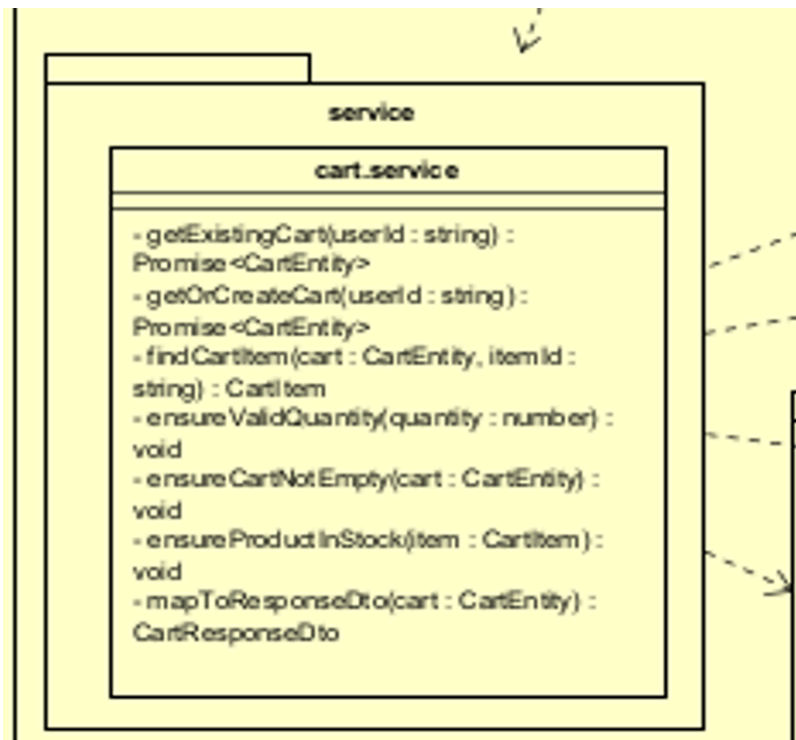


Table 2. Operation design

#	Name	Return type	Description (purpose)
1	- getExistingCart(userId)	Promise<CartEntity>	Load an existing cart for a user. Used when the operation must fail if the cart is missing.
2	- getOrCreateCart(userId)	Promise<CartEntity>	Return the current cart if it exists; otherwise create a new cart aggregate for the given user.
3	- findCartItem(cart, itemId)	CartItem	Locate a line item inside the aggregate by its identifier.
4	-ensureValidQuantity(quantity)	void	Validate quantity rules before a cart mutation is applied.
5	- ensureCartNotEmpty(cart)	void	Guard operation used before validation or checkout-related processing.
6	- ensureProductInStock(item)	void	Check whether the referenced product line is still fulfillable according to stock constraints.
7	- mapToResponseDto(cart)	CartResponseDto	Transform the aggregate root into a response DTO for the upper layer.

Parameter:

- `userId` (string): user identity used for repository queries and cart ownership checks.
- `cart: CartEntity`: current aggregate root used by internal helper methods.
- `itemId` (string): line-item identifier used during search.
- `quantity` (number): requested quantity that must satisfy positive numeric constraints.
- `item: CartItem`: target line item whose product availability must be checked.

Exception:

- `CartNotFoundException` from `getExistingCart()` when a mandatory cart is absent.
- `CartItemNotFoundException` from `findCartItem()` when the requested item is missing.
- `InvalidCartItemQuantityException` from `ensureValidQuantity()` when quantity violates input rules.
- `EmptyCartException` from `ensureCartNotEmpty()` when the aggregate has no line items.
- `ProductOutOfStockException` from `ensureProductInStock()` when stock is insufficient.

Method

This class coordinates the cart business rules. It should retrieve the aggregate from `cartRepo`, enforce guards, update derived totals, and map domain data to boundary DTOs. Validation logic is intentionally centralized here to keep the controller thin and the repository persistence-focused.

Other diagrams

- No additional diagram is required.

Design for class “Cart.entity”

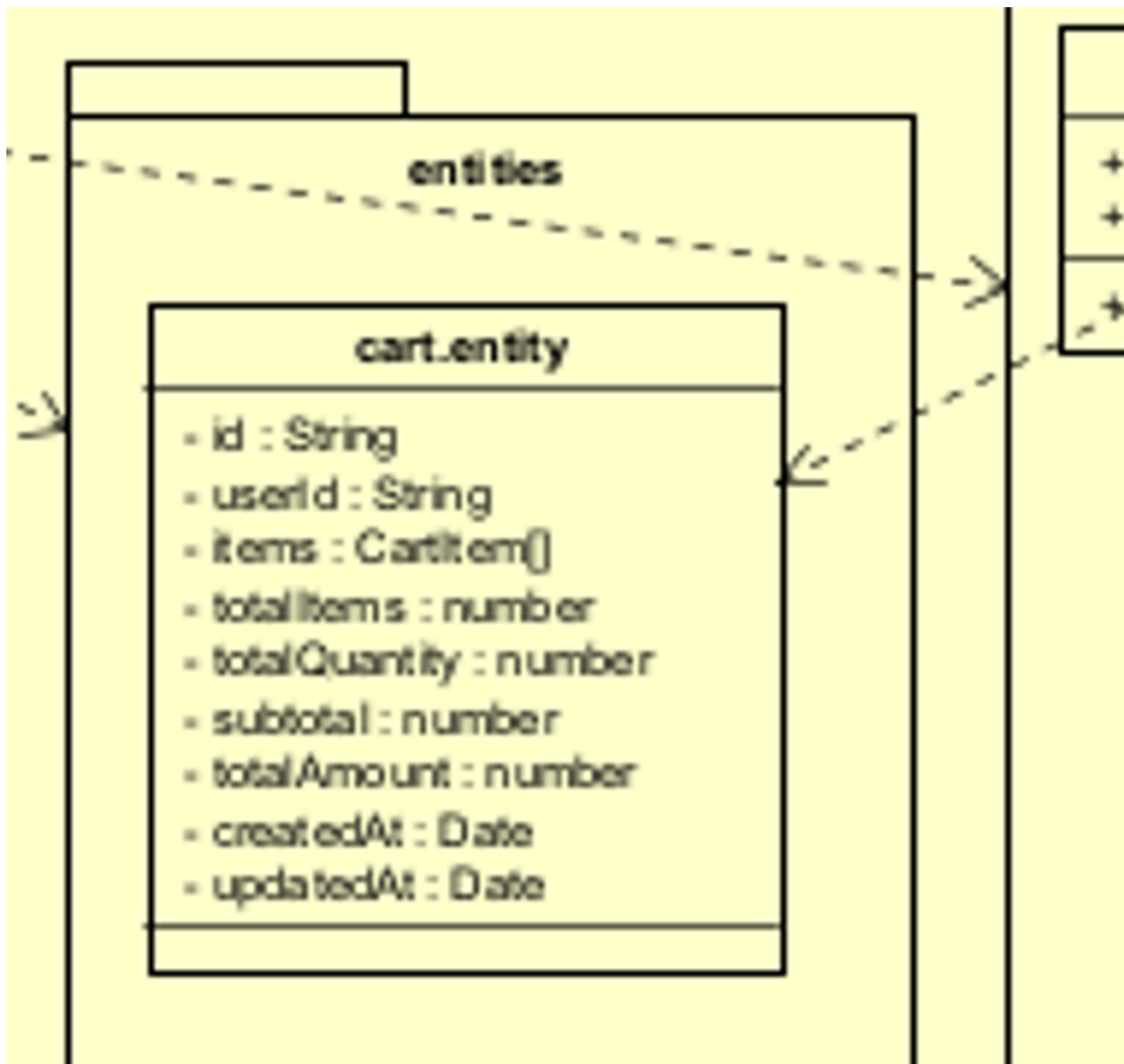


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	- id	String	""	Unique identifier of the cart aggregate root.
2	- userId	String	""	Identifier of the user who owns the cart.
3	- items	CartItem[]	[]	Collection of cart line items currently contained in the cart.

4	- totalItems	number	0	Derived count of distinct cart lines.
5	- totalQuantity	number	0	Derived sum of the quantities of all cart lines.
6	- subtotal	number	0	Monetary sum before any additional charge or final pricing rule.
7	- totalAmount	number	0	Final amount currently payable for the cart.
8	- createdAt	Date	current timestamp	Creation timestamp of the aggregate.
9	- updatedAt	Date	current timestamp	Last update timestamp of the aggregate.

Parameter:

- No explicit operation parameters were modeled for this class in the provided diagram.

Exception:

- No class-specific exception is thrown directly by the entity in the current model; business guards are handled by the service layer.

Method

`cart.entity` stores the persistent and derived state of the cart aggregate. The service layer is responsible for mutating items and recomputing `totalItems`, `totalQuantity`, `subtotal`, and `totalAmount` whenever the aggregate changes.

Other diagrams

- No additional diagram is required.

Design for class “cart-item-list.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	cartItems	CartItemModel[]	[]	Stores the cart items currently displayed in the item list.

2	insufficientStockMessages	string[]	[]	Stores warning messages for products whose stock is insufficient.
3	totalDisplayedItems	number	0	Stores the number of cart items shown on screen.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	displayCartItems()	void	Renders the current cart items on the cart page.
2	changeItemQuantity(productId : string, quantity: number)	void	Requests a quantity change for a selected cart item.
3	removeItem(productId: string)	void	Removes a selected cart item from the list.

Parameter:

- changeItemQuantity.productId: none, Identifier of the product whose quantity is changed.
- changeItemQuantity.quantity: none, New quantity requested by the customer.
- removeItem.productId: none, Identifier of the product to remove from the cart.

Exception:

- ValidationException if quantity is less than 1.
- StockException if the requested quantity exceeds the available stock.

Method:

- Loads cart items from cart.service and displays them in list form.
- Whenever quantity changes, it forwards the request to cart.service and refreshes the list.
- Shows stock warnings returned by the cart validation result.

Design for class “cart-page.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	cart	CartModel	new CartModel()	Stores the current cart shown on the page.
2	isStockValid	boolean	true	Indicates whether all requested quantities are currently satisfiable.
3	canProceedToDelivery	boolean	false	Indicates whether the customer can move to the delivery step.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadCart()	void	Loads the current cart data when the page is opened.
2	refreshCartStatus()	void	Refreshes stock validation and checkout readiness.
3	goToDeliveryPage()	void	Navigates to the delivery step if the cart is valid.

Parameter:

- None.

Exception:

- StockException if one or more products in the cart are no longer sufficiently available.

Method:

- Opens with the latest cart information, including total price excluding VAT.
- Checks whether the cart still satisfies the stock rules before checkout.
- Allows navigation to the delivery page only after successful validation.

Design for class “cart-summary.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	totalItems	number	0	Stores the total quantity of products in the cart.
2	totalPriceExclVAT	number	0	Stores the total cart value excluding VAT.
3	warningCount	number	0	Stores the number of stock warnings currently shown.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	updateSummary(cart: CartModel)	void	Updates the summary values from the current cart.
2	displayStockWarnings(cart: CartModel)	void	Displays stock warning messages for insufficient items.

Parameter:

- updateSummary.cart: none, Cart used to compute summary values.
- displayStockWarnings.cart: none, Cart used to derive stock warning messages.

Exception:

- No business exception; this class only presents computed values.

Method:

- Reads the cart model and shows the total quantity and total price excluding VAT.
- Displays warnings when cart items cannot be fully fulfilled.

Design for class “cart.service”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	cart	CartModel	new CartModel()	Stores the current cart state shared across the cart step.
2	stockWarningCount	number	0	Stores the number of items currently having stock issues.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadCart()	CartModel	Returns the current cart for display or update.
2	addProduct(item: CartItemModel)	void	Adds a product with quantity information into the cart.
3	removeProduct(productId: string)	void	Removes a product from the cart.
4	changeProductQuantity(productId : string, quantity: number)	void	Changes the quantity of a product already in the cart.
5	validateCartStock()	boolean	Checks whether all cart quantities can still be fulfilled.
6	clearCart()	void	Clears all cart items after a completed order or cancellation.

Parameter:

- addProduct.item: none, Cart item object to add into the current cart.
- removeProduct.productId: none, Identifier of the item to remove.
- changeProductQuantity.productId: none, Identifier of the item to update.
- changeProductQuantity.quantity: none, Requested new quantity.

Exception:

-
- ValidationException if quantity is invalid.
 - StockException if requested quantity is greater than available stock.

Method:

- Maintains the authoritative cart state for all cart-related components.
- Validates stock before customers proceed to the delivery step.
- Recomputes cart totals whenever items are added, removed, or updated.

Design for class “cart-item.model”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	productId	string	""	Unique identifier of the product.
2	title	string	""	Name of the product displayed to the customer.
3	category	string	""	Product category used for display and filtering.
4	unitPrice	number	0	Current product price excluding VAT.
5	unitWeight	number	0	Weight of a single product unit used for delivery-fee calculation.
6	quantity	number	1	Quantity requested by the customer.
7	stockAvailable	number	0	Current stock quantity available for ordering.
8	missingQuantity	number	0	Quantity still lacking when stock is insufficient.
9	lineTotal	number	0	Total amount for this line excluding VAT.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	calculateLineTotal()	number	Calculates the total amount of the item line excluding VAT.
2	calculateMissingQuantity()	number	Calculates how many units are lacking compared with requested quantity.
3	isStockSufficient()	boolean	Checks whether current stock is enough for the requested quantity.

Parameter:

- None.

Exception:

- ValidationException if quantity or price is negative.

Method:

- Represents one product row in the cart and invoice flows.
- Provides helper calculations for amount and stock warnings.

Design for class “cart.model”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	items	CartItemModel[]	[]	Stores all products currently selected by the customer.
2	totalItems	number	0	Stores the total quantity of products in the cart.

3	totalPriceExclVAT	number	0	Stores the total amount of all cart items excluding VAT.
4	totalWeight	number	0	Stores the total shipment weight derived from all cart items.
5	stockValid	boolean	true	Indicates whether the entire cart satisfies the stock condition.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	addItem(item: CartItemModel)	void	Adds a new item into the cart.
2	removeItem(productId: string)	void	Removes an item from the cart.
3	updateItemQuantity(productId: string, quantity: number)	void	Updates the quantity of an existing item.
4	calculateTotalPriceExclVAT()	number	Calculates the cart total excluding VAT.
5	calculateTotalWeight()	number	Calculates the cart total shipment weight.

Parameter:

- addItem.item: none, Item to add into the cart.
- removeItem.productId: none, Identifier of the item to remove.
- updateItemQuantity.productId: none, Identifier of the item to update.
- updateItemQuantity.quantity: none, New requested quantity.

Exception:

- ValidationException if quantity is not valid.
- StockException if updated quantity is greater than available stock.

Method:

- Aggregates all cart items selected by the customer.
- Provides summary values required by cart, delivery, and invoice steps.

Design for class “delivery-info-field.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	fieldName	string	""	Logical name of the delivery field, such as recipientName or phoneNumber.
2	fieldValue	string	""	Current value entered by the customer.
3	isRequired	boolean	true	Indicates whether the field is mandatory.
4	errorMessage	string	""	Validation message displayed when the field value is invalid.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	setValue(value: string)	void	Updates the current field value.
2	validateField()	boolean	Checks whether the field value satisfies required rules.
3	clearError()	void	Clears the current validation message.

Parameter:

- setValue.value: none, New field value entered by the customer.

Exception:

- ValidationException if a required field is left blank or has invalid format.

Method:

-
- Represents one editable field in the delivery form.
 - Validates user input before the delivery step can continue.

Design for class “delivery-method-selector.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	selectedMethod	string	"standard"	Stores the delivery method currently selected by the customer.
2	availableMethods	string[]	[]	Stores the delivery methods currently offered by the system.
3	deliveryFee	number	0	Stores the fee shown for the currently selected method.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	selectMethod(method: string)	void	Changes the currently selected delivery method.
2	displayDeliveryFee(fee: number)	void	Updates the displayed delivery fee.
3	emitMethodChange()	void	Notifies the delivery page that the method has changed.

Parameter:

- selectMethod.method: none, Delivery method chosen by the customer.
- displayDeliveryFee.fee: none, Delivery fee to display for the chosen method.

Exception:

- ValidationException if the selected delivery method is unsupported.

Method:

- Lets the customer select or change the delivery method.
- Triggers delivery-fee recalculation when the method changes.

Design for class “delivery-page.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	deliveryInfo	DeliveryInfoModel	new DeliveryInfoModel()	Stores the delivery information currently edited on the page.
2	cart	CartModel	new CartModel()	Stores the current cart needed for fee recalculation.
3	invoicePreview	InvoiceInfoModel	new InvoiceInfoModel()	Stores the invoice preview refreshed after delivery changes.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadDeliveryInfo()	void	Loads existing delivery information when the page opens.
2	recalculateDeliveryFee()	void	Recalculates the fee after address or method changes.
3	proceedToPayment()	void	Navigates to the payment step after successful validation.

Parameter:

- None.

Exception:

- ValidationException if required delivery information is missing or invalid.

Method:

- Displays delivery information together with cart information.
- Validates the form and recalculates delivery fee whenever address or method changes.
- Updates the invoice preview before moving to payment.

Design for class “delivery-info.model”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	recipientName	string	""	Customer name used for order delivery.
2	phoneNumber	string	""	Customer phone number used for contact.
3	email	string	""	Customer email used to receive invoice and transaction information.
4	streetAddress	string	""	Detailed delivery address.
5	province	string	""	Province or city used in shipping-fee rules.
6	deliveryMethod	string	"standard"	Chosen delivery method.
7	deliveryFee	number	0	Current delivery fee for the order.

8	isValid	boolean	false	Indicates whether all required delivery information is valid.
---	---------	---------	-------	---

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validate()	boolean	Checks whether all required fields are complete and valid.
2	updateAddress(streetAddress: string, province: string)	void	Updates the delivery address information.
3	setDeliveryFee(fee: number)	void	Updates the current delivery fee.

Parameter:

- updateAddress.streetAddress: none, Detailed shipping address.
- updateAddress.province: none, Province or city for the address.
- setDeliveryFee.fee: none, Computed delivery fee.

Exception:

- ValidationException if address, phone number, or email format is invalid.

Method:

- Stores the delivery information collected before payment.
- Supports validation and fee updates when customers revise their address.

Design for class “delivery.service”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	currentDeliveryInfo	DeliveryInfoModel	new DeliveryInfoModel()	Stores the delivery information currently being processed.

2	freeShippingThreshold	number	100000	Order value threshold for free-shipping discount consideration.
3	freeShippingCap	number	25000	Maximum free-shipping amount per order.
4	cityBaseFee	number	22000	Base fee for orders delivered in Ha Noi or Ho Chi Minh City.
5	provinceBaseFee	number	30000	Base fee for orders delivered elsewhere in Vietnam.
6	extraHalfKgFee	number	2500	Additional fee charged for each 0.5kg beyond the base tier.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	validateDeliveryInfo(info: DeliveryInfoModel)	boolean	Validates delivery information before payment.
2	calculateDeliveryFee(totalWeight: number, province: string, totalItemValue: number)	number	Calculates the applicable delivery fee for the order.
3	applyFreeShipping(totalItemValue: number, deliveryFee: number)	number	Applies the free-shipping discount when eligible.
4	saveDeliveryInfo(info: DeliveryInfoModel)	void	Stores the latest delivery information.

Parameter:

- validateDeliveryInfo.info: none, Delivery information to validate.

-
- calculateDeliveryFee.totalWeight: none, Total shipment weight of the order.
 - calculateDeliveryFee.province: none, Province or city used in fee rules.
 - calculateDeliveryFee.totalItemValue: none, Order value excluding shipping for free-shipping evaluation.
 - applyFreeShipping.totalItemValue: none, Order value used to check free-shipping eligibility.
 - applyFreeShipping.deliveryFee: none, Calculated shipping fee before discount.
 - saveDeliveryInfo.info: none, Delivery information to persist.

Exception:

- ValidationException if delivery information is incomplete or invalid.
- BusinessException if fee rules cannot be applied because mandatory inputs are missing.

Method:

- Implements the AIMS shipping-fee rules based on address, weight, and free-shipping condition.
- Recalculates delivery fee each time the address or delivery method changes.

Design for class “invoice-item-list.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	items	CartItemModel[]	[]	Stores the invoice items displayed on the invoice page.
2	totalItemCount	number	0	Stores the number of item rows currently shown.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadItems(invoiceInfo: InvoiceInfoModel)	void	Loads invoice items from the current invoice model.
2	renderItems()	void	Displays invoice items and their quantities on screen.

Parameter:

- loadItems.invoiceInfo: none, Invoice information used to populate the item list.

Exception:

- No business exception; this class only presents invoice items.

Method:

- Displays the products, quantities, and prices included in the invoice preview.

Design for class “invoice-page.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	invoiceInfo	InvoiceInfoModel	new InvoiceInfoModel()	Stores the invoice currently displayed to the customer.
2	canProceedToPayment	boolean	false	Indicates whether payment can start with the current invoice.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadInvoice()	void	Loads the temporary invoice when the invoice page opens.
2	refreshInvoice()	void	Refreshes invoice values when cart or delivery data changes.
3	proceedToPayment()	void	Navigates to the payment step.

Parameter:

- None.

Exception:

-
- ValidationException if invoice data is incomplete or inconsistent.

Method:

- Displays the temporary invoice before payment.
- Refreshes totals after cart or delivery changes and then allows payment.

Design for class “invoice-summary.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	totalProductPriceExclVAT	number	0	Stores the total product price excluding VAT.
2	totalProductPriceInclVAT	number	0	Stores the total product price including VAT.
3	deliveryFee	number	0	Stores the delivery fee.
4	totalAmount	number	0	Stores the final total amount to be paid.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	bindInvoice(invoiceInfo: InvoiceInfoModel)	void	Copies summary values from the invoice model.
2	displaySummary()	void	Displays the invoice summary to the customer.

Parameter:

- bindInvoice.invoiceInfo: none, Invoice information that provides summary values.

Exception:

- No business exception; this class only displays computed summary values.

Method:

- Shows the invoice totals, including VAT, delivery fee, and final payable amount.

Design for class “invoice-info.model”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	items	CartItemModel[]	[]	Stores the products included in the invoice.
2	totalProductPriceExclVAT	number	0	Total product amount excluding VAT.
3	vatAmount	number	0	VAT amount computed from product totals.
4	totalProductPriceInclVAT	number	0	Total product amount including VAT.
5	deliveryFee	number	0	Shipping fee not subject to tax.
6	totalAmount	number	0	Final amount payable by the customer.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	calculateVAT()	number	Calculates the VAT amount for the product total.
2	calculateTotalAmount()	number	Calculates the final amount to pay.
3	refresh(cart: CartModel, deliveryInfo: DeliveryInfoModel)	void	Refreshes the invoice from the current cart and delivery information.

Parameter:

- refresh.cart: none, Current cart containing ordered items.
- refresh.deliveryInfo: none, Delivery information providing the delivery fee.

Exception:

- ValidationException if invoice inputs are missing or invalid.

Method:

- Temporarily stores invoice information before payment.
- Combines cart totals, VAT, and delivery fee into the final payable amount.

Design for class “invoice.service”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	currentInvoice	InvoiceInfoModel	new InvoiceInfoModel()	Stores the latest invoice generated during checkout.
2	vatRate	number	0.1	VAT rate applied to product prices.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	createInvoice(cart: CartModel, deliveryInfo: DeliveryInfoModel)	InvoiceInfoModel	Creates a temporary invoice from cart and delivery information.
2	updateInvoice(cart: CartModel, deliveryInfo: DeliveryInfoModel)	InvoiceInfoModel	Refreshes invoice values when checkout data changes.
3	calculateProductPriceInclVAT(totalPriceExclVAT: number)	number	Calculates the total product

			price including VAT.
--	--	--	----------------------

Parameter:

- createInvoice.cart: none, Current cart of selected products.
- createInvoice.deliveryInfo: none, Delivery information containing the fee.
- updateInvoice.cart: none, Current cart after modifications.
- updateInvoice.deliveryInfo: none, Updated delivery information.
- calculateProductPriceInclVAT.totalPriceExclVAT: none, Product amount before VAT.

Exception:

- ValidationException if invoice source data is incomplete.

Method:

- Creates and updates the temporary invoice shown before payment.
- Ensures VAT applies only to product prices, while delivery fee remains untaxed.

Design for class “payment-page.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	invoiceInfo	InvoiceInfoModel	new InvoiceInfoModel()	Stores the invoice being paid.
2	selectedPaymentMethod	string	"vietqr"	Stores the payment method currently selected by the customer.
3	transactionInfo	OrderTransactionInfoModel	new OrderTransactionInfoModel()	Stores the transaction generated for the payment attempt.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadPaymentPage()	void	Loads invoice and default payment information.
2	changePaymentMethod(method: string)	void	Changes the current payment method.
3	submitPayment()	void	Starts the payment process with the selected method.

Parameter:

- changePaymentMethod.method: none, Selected payment method, such as vietqr or paypal.

Exception:

- PaymentException if the selected gateway cannot generate a payment request.

Method:

- Displays the default QR-code payment and allows switching to PayPal.
- Starts payment after invoice review is complete.

Design for class “payment-section.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	qrCodeContent	string	""	Stores the QR content or image URL displayed for VietQR payment.
2	alternateGatewayLabel	string	"PayPal"	Stores the label of the alternative payment gateway.
3	paymentMessage	string	""	Stores the payment status message shown to the customer.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	showVietQR(transactionInfo: OrderTransactionInfoModel)	void	Displays the VietQR payment information.
2	showPayPal()	void	Displays the PayPal payment option.
3	showPaymentStatus()	void	Displays the current payment result message.

Parameter:

- showVietQR.transactionInfo: none, Transaction information used to render the QR payment block.

Exception:

- PaymentException if no gateway-specific content is available for display.

Method:

- Renders payment-specific UI content for VietQR and PayPal.
- Shows messages after payment succeeds, fails, or remains pending.

Design for class “order-transaction-info.model”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	transactionId	string	""	Unique identifier of the payment transaction.
2	transactionContent	string	""	Payment content or note associated with the transaction.
3	transactionDateTime	Date	null	Date and time when the transaction result is recorded.
4	paymentMethod	string	"vietqr"	Method used for the transaction.
5	amountPaid	number	0	Amount actually paid by the customer.

6	paymentStatus	string	"pending"	Current transaction state, such as pending, paid, or failed.
7	gatewayReference	string	""	Gateway-specific reference returned by VietQR or PayPal.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	markPending()	void	Sets the transaction state to pending.
2	markPaid()	void	Sets the transaction state to paid.
3	markFailed()	void	Sets the transaction state to failed.

Parameter:

- None.

Exception:

- ValidationException if transaction state changes are inconsistent with recorded payment data.

Method:

- Stores the payment transaction information recorded after checkout.

Design for class “payment.service”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	selectedPaymentMethod	string	"vietqr"	Stores the current payment method used in checkout.
2	currentTransaction	OrderTransactionInfoModel	new OrderTransactionInfoModel()	Stores the latest payment transaction.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	createVietQRTransaction(invoiceInfo: InvoiceInfoModel)	OrderTransactionInfoModel	Generates a VietQR payment request and transaction record.
2	createPayPalTransaction(invoiceInfo: InvoiceInfoModel)	OrderTransactionInfoModel	Generates a PayPal payment request and transaction record.
3	switchPaymentMethod(method: string)	void	Changes the active payment method.
4	verifyPaymentResult(transactionId: string)	boolean	Checks whether the payment result is successful.

Parameter:

- createVietQRTransaction.invoiceInfo: none, Invoice information used to create the VietQR payment.
- createPayPalTransaction.invoiceInfo: none, Invoice information used to create the PayPal payment.
- switchPaymentMethod.method: none, Chosen payment gateway.
- verifyPaymentResult.transactionId: none, Identifier of the transaction to verify.

Exception:

- PaymentException if gateway communication fails or returns an invalid response.
- ValidationException if the selected payment method is unsupported.

Method:

-
- Creates payment requests for VietQR and PayPal.
 - Stores and verifies the payment transaction result before the order-success step.

Design for class “successful-order-info.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	order	SuccessfulOrderModel	new SuccessfulOrderModel()	Stores the successful order currently displayed.
2	displayMode	string	"summary"	Controls whether the component shows summary or detailed content.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	bindOrder(order: SuccessfulOrderModel)	void	Loads successful order data into the component.
2	showTransactionInfo()	void	Displays the transaction section of the order result.

Parameter:

- bindOrder.order: none, Successful order data to display.

Exception:

- No business exception; this class only presents order-success information.

Method:

- Displays customer, shipping, and transaction information after successful payment.

Design for class “successful-page.component”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	order	SuccessfulOrderModel	new SuccessfulOrderModel()	Stores the completed order shown on the success page.
2	emailSent	boolean	false	Indicates whether confirmation information has already been emailed.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	loadOrderResult()	void	Loads the final order and transaction information.
2	navigateToOrderDetails()	void	Opens the detailed order view using the available link.

Parameter:

- None.

Exception:

- ValidationException if required order or transaction information is missing.

Method:

- Shows the final successful-order page after payment completion.
- Provides access to order details through the stored order link.

Design for class “successful-order.model”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	orderId	string	""	Unique identifier of the successful order.
2	customerName	string	""	Customer name shown in the success result.
3	phoneNumber	string	""	Customer phone number for the order.
4	shippingAddress	string	""	Shipping address of the order.
5	province	string	""	Province or city of the shipping address.
6	totalAmount	number	0	Final amount paid by the customer.
7	transactionId	string	""	Identifier of the payment transaction.
8	transactionContent	string	""	Content or note of the payment transaction.
9	transactionDateTime	Date	null	Date and time of the recorded transaction.
10	orderStatus	string	"pending processing"	Initial status of the order after successful payment.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	buildFrom(deliveryInfo: DeliveryInfoModel, invoiceInfo: InvoiceInfoModel, transactionInfo: OrderTransactionInfoModel)	void	Builds the successful order from checkout and payment data.
2	canCancelBeforeApproval()	boolean	Checks whether the order can still be cancelled before approval.

Parameter:

- buildFrom.deliveryInfo: none, Delivery information of the completed order.
- buildFrom.invoiceInfo: none, Final invoice information.
- buildFrom.transactionInfo: none, Recorded payment transaction information.

Exception:

- ValidationException if any mandatory success-order field is missing.

Method:

- Aggregates the order result shown after payment succeeds.
- Keeps the order in pending processing state until reviewed by Product Manager.

Design for class “success-order.service”

<Paste UML class image here>

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	currentSuccessfulOrder	SuccessfulOrderModel	new SuccessfulOrderModel()	Stores the current successful order after payment completion.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	createSuccessfulOrder(deliveryInfo: DeliveryInfoModel, invoiceInfo: InvoiceInfoModel, transactionInfo: OrderTransactionInfoModel)	SuccessfulOrderModel	Creates the successful-order result from delivery, invoice, and transaction data.
2	sendOrderEmail(order: SuccessfulOrderModel)	void	Sends invoice and transaction information to the customer email.
3	getSuccessfulOrder()	SuccessfulOrderModel	Returns the current successful-order information.

Parameter:

- createSuccessfulOrder.deliveryInfo: none, Delivery information of the order.
- createSuccessfulOrder.invoiceInfo: none, Final invoice information.
- createSuccessfulOrder.transactionInfo: none, Transaction information recorded after payment.
- sendOrderEmail.order: none, Successful order whose confirmation email will be sent.

Exception:

- NotificationException if confirmation email cannot be sent.
- ValidationException if successful-order data is incomplete.

Method:

- Creates the success-order aggregate after payment succeeds.
- Triggers confirmation email delivery and exposes the final order data to the success page.

